NCA

October 8, 2025

[5]:
```
# Cell 1 - Setup & load data
import os
from pathlib import Path
import numpy as np
import pandas as pd

ROOT = Path.cwd()
DATA_RAW = ROOT / "data" / "raw"
DATA_PROCESSED = ROOT / "data" / "processed"
EXPERIMENTS_OUT = DATA_PROCESSED / "experiments"
EXPERIMENTS_OUT.mkdir(parents=True, exist_ok=True)

# Load expression (neurons x genes, columns are WBGene IDs)
expr_path = DATA_PROCESSED / "csv" / "gene_expr_CeNGEN_base_mapped.csv"
expr_df = pd.read_csv(expr_path, index_col=0)

# Load connectome edges (Source, Target, SynapseCount)
edges_path = DATA_PROCESSED / "csv" / "merged_edges_with_relatedness.csv"
edges_df = pd.read_csv(edges_path)
edges_df.columns = [c.lower() for c in edges_df.columns]

src_col = [c for c in edges_df.columns if "source" in c][0]
tgt_col = [c for c in edges_df.columns if "target" in c][0]
w_col   = "synapsecount" if "synapsecount" in edges_df.columns else None

print(f"expr_df: {expr_df.shape}; edges: {edges_df.shape}; weight_col: {w_col}")
```

```
expr_df: (838, 22469); edges: (27727, 6); weight_col: synapsecount
```

[6]:
```
# Cell 2 - Graph, features, and targets
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Keep only neurons present in expression
neurons = expr_df.index.tolist()
neuron_to_idx = {n:i for i,n in enumerate(neurons)}
N = len(neurons)
```

```
# Build (directed) adjacency list from edges; drop edges where nodes not in␣
↪expr_df
mask_src = edges_df[src_col].isin(neurons)
mask_tgt = edges_df[tgt_col].isin(neurons)
Edf = edges_df[mask_src & mask_tgt].copy()

# Edge weights
if w_col is None:
    Edf["w"] = 1.0
else:
    Edf["w"] = Edf[w_col].astype(float).clip(lower=0.0)

# Sparse adjacency in COO form
rows = Edf[src_col].map(neuron_to_idx).values
cols = Edf[tgt_col].map(neuron_to_idx).values
wts  = Edf["w"].values.astype(np.float32)

# Target: mean outgoing synaptic strength per neuron (normalize)
out_w = Edf.groupby(src_col)["w"].mean().reindex(neurons).fillna(0.0).values.
↪astype(np.float32)
y = (out_w - out_w.mean()) / (out_w.std() + 1e-6)  # z-score target

# ==== Features ====
USE_PCA = True        # set False to use raw subset of genes
N_PCA   = 32          # PCA dims if USE_PCA=True
RAW_K   = 512         # top-variance genes if USE_PCA=False

if USE_PCA:
    scaler = StandardScaler(with_mean=True, with_std=True)
    X_scaled = scaler.fit_transform(expr_df.values)         # (N x G)
    pca = PCA(n_components=min(N_PCA, X_scaled.shape[1]), random_state=42)
    X_feat = pca.fit_transform(X_scaled).astype(np.float32) # (N x d)
else:
    # raw features: top-variance genes
    top_genes = expr_df.var(axis=0).sort_values(ascending=False).head(RAW_K).
↪index
    X_feat = expr_df.loc[:, top_genes].fillna(0.0).values.astype(np.float32)
    # z-score per feature
    X_feat = (X_feat - X_feat.mean(0)) / (X_feat.std(0) + 1e-6)

print(f"N={N}, edges kept={len(rows)}, features={X_feat.shape}, target y␣
↪shape={y.shape}")
```

```
N=838, edges kept=16673, features=(838, 32), target y shape=(838,)
```

[7]:
```
# Cell 3 - Graph Neural Cellular Automata (GNCA)
import torch
```

```python
import torch.nn as nn
import torch.nn.functional as F

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("device:", device)

# Convert arrays to torch
X_feat_t = torch.from_numpy(X_feat).to(device)              # (N, d_in)
y_t      = torch.from_numpy(y).to(device)                   # (N,)

# Build sparse adjacency (row-normalized for message passing)
# We'll use outgoing adjacency (source -> target)
indices = torch.stack([
    torch.from_numpy(rows).long(),
    torch.from_numpy(cols).long()
], dim=0).to(device)
values  = torch.from_numpy(wts).to(device)
A = torch.sparse_coo_tensor(indices, values, size=(N, N)).coalesce()

# Row-normalize A
row_sum = torch.sparse.sum(A, dim=1).to_dense().clamp_min(1e-6)  # (N,)
row_inv = 1.0 / row_sum
row_inv_diag = torch.sparse_coo_tensor(
    torch.stack([torch.arange(N), torch.arange(N)]).to(device),
    row_inv, size=(N, N)
).coalesce()
A_norm = torch.sparse.mm(row_inv_diag, A).coalesce()  # normalized by␣
 ↪out-degree weight sum

class GNCA(nn.Module):
    def __init__(self, in_dim, state_dim=64, fire_rate=0.5):
        super().__init__()
        self.state_dim  = state_dim
        self.fire_rate  = fire_rate
        # init embed from features to state
        self.inp = nn.Linear(in_dim, state_dim)
        # message and update (residual)
        self.msg = nn.Linear(state_dim, state_dim, bias=False)
        self.upd = nn.Sequential(
            nn.Linear(2*state_dim, 128),
            nn.ReLU(inplace=True),
            nn.Linear(128, state_dim)
        )
        # readout head to scalar
        self.readout = nn.Sequential(
            nn.LayerNorm(state_dim),
            nn.Linear(state_dim, 1)
```

```
        )

    def forward(self, x_feat, A_norm, steps=8, train_mode=True):
        # initialize state from features
        x = torch.tanh(self.inp(x_feat))   # (N, C)
        for t in range(steps):
            # messages from neighbors: A_norm @ msg(x)
            m = torch.sparse.mm(A_norm, self.msg(x))
            # concat self-state and neighbor-message
            z = torch.cat([x, m], dim=-1)
            dx = self.upd(z)
            # stochastic "fire" mask for asynch updates
            if train_mode and self.fire_rate < 1.0:
                mask = (torch.rand_like(dx[:, :1]) < self.fire_rate).float()
                dx = dx * mask
            # residual update
            x = x + dx
        # readout scalar per node
        yhat = self.readout(x).squeeze(-1)
        return yhat, x

model = GNCA(in_dim=X_feat_t.shape[1], state_dim=64, fire_rate=0.5).to(device)
sum(p.numel() for p in model.parameters())/1e6
```

```
device: cuda
```

[7]: 0.031169

[8]:
```
# Cell 4 - Train GNCA to predict outgoing synaptic strength
import math
torch.manual_seed(42)

optim = torch.optim.AdamW(model.parameters(), lr=3e-3, weight_decay=1e-4)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(optim, T_max=200,␣
 ↪eta_min=3e-4)
loss_fn = nn.MSELoss()

EPOCHS = 200
STEPS  = 8  # unroll steps

best = math.inf
for ep in range(1, EPOCHS+1):
    model.train()
    yhat, _ = model(X_feat_t, A_norm, steps=STEPS, train_mode=True)
    loss = loss_fn(yhat, y_t)
    optim.zero_grad()
    loss.backward()
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)
```

```
    optim.step()
    sched.step()

    if loss.item() < best:
        best = loss.item()
        torch.save(model.state_dict(), EXPERIMENTS_OUT / "gnca_best.pt")

    if ep % 20 == 0 or ep == 1:
        with torch.no_grad():
            # Pearson r
            ycpu = y_t.detach().cpu().numpy()
            phat = yhat.detach().cpu().numpy()
            r = np.corrcoef(ycpu, phat)[0,1]
        print(f"Epoch {ep:03d} | loss={loss.item():.4f} | r={r:.3f}")

print("Best loss:", best)
```

```
Epoch 001 | loss=1.0821 | r=0.145
Epoch 020 | loss=0.1917 | r=0.899
Epoch 040 | loss=0.1005 | r=0.951
Epoch 060 | loss=0.0733 | r=0.966
Epoch 080 | loss=0.0472 | r=0.976
Epoch 100 | loss=0.0486 | r=0.978
Epoch 120 | loss=0.0294 | r=0.985
Epoch 140 | loss=0.0285 | r=0.986
Epoch 160 | loss=0.0283 | r=0.986
Epoch 180 | loss=0.0220 | r=0.989
Epoch 200 | loss=0.0240 | r=0.988
Best loss: 0.01777643710374832
```

[9]:
```
# Cell 5 - Final eval & export
model.load_state_dict(torch.load(EXPERIMENTS_OUT / "gnca_best.pt",␣
 ↪map_location=device))
model.eval()
with torch.no_grad():
    yhat, X_final = model(X_feat_t, A_norm, steps=8, train_mode=False)
    yhat = yhat.detach().cpu().numpy()
    ytrue = y_t.detach().cpu().numpy()
    r = np.corrcoef(ytrue, yhat)[0,1]
    mae = np.abs(ytrue - yhat).mean()
print(f"Final Pearson r={r:.3f} | MAE={mae:.3f}")

pred_df = pd.DataFrame({
    "neuron": neurons,
    "y_true_z": ytrue,
    "y_pred_z": yhat
})
```

```
pred_path = EXPERIMENTS_OUT / "gnca_pred_outgoing_strength.csv"
pred_df.to_csv(pred_path, index=False)
print("Saved:", pred_path)
```

```
Final Pearson r=0.987 | MAE=0.097
Saved: /mnt/ssd4tb/Desktop/C-
Elegans/data/processed/experiments/gnca_pred_outgoing_strength.csv
```

```
/tmp/ipykernel_150307/766371716.py:2: FutureWarning: You are using `torch.load`
with `weights_only=False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malicious pickle data
which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for
more details). In a future release, the default value for `weights_only` will be
flipped to `True`. This limits the functions that could be executed during
unpickling. Arbitrary objects will no longer be allowed to be loaded via this
mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting
`weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this
experimental feature.
  model.load_state_dict(torch.load(EXPERIMENTS_OUT / "gnca_best.pt",
map_location=device))
```

[10]:
```
# Cell A - Shuffle control: permute expression rows; structure should vanish (r␣
↪~ 0)
import torch, numpy as np
torch.manual_seed(123)

perm = torch.randperm(X_feat_t.shape[0], device=X_feat_t.device)
X_feat_shuf = X_feat_t[perm]  # break neuron-expression alignment

model_shuf = GNCA(in_dim=X_feat_t.shape[1], state_dim=64, fire_rate=0.5).
↪to(device)
opt = torch.optim.AdamW(model_shuf.parameters(), lr=3e-3, weight_decay=1e-4)
sch = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=80, eta_min=5e-4)
loss_fn = torch.nn.MSELoss()

EPOCHS = 80
for ep in range(1, EPOCHS+1):
    model_shuf.train()
    yhat, _ = model_shuf(X_feat_shuf, A_norm, steps=8, train_mode=True)
    loss = loss_fn(yhat, y_t)
    opt.zero_grad(); loss.backward(); torch.nn.utils.clip_grad_norm_(model_shuf.
↪parameters(), 1.0); opt.step(); sch.step()
    if ep % 20 == 0 or ep == 1:
        with torch.no_grad():
```

```
            r = np.corrcoef(y_t.detach().cpu().numpy(), yhat.detach().cpu().
 ↪numpy())[0,1]
        print(f"[SHUF] Epoch {ep:03d} | loss={loss.item():.4f} | r={r:.3f}")
```

```
[SHUF] Epoch 001 | loss=1.1639 | r=0.084
[SHUF] Epoch 020 | loss=0.2437 | r=0.872
[SHUF] Epoch 040 | loss=0.1874 | r=0.901
[SHUF] Epoch 060 | loss=0.1509 | r=0.922
[SHUF] Epoch 080 | loss=0.1297 | r=0.934
```

[11]:
```
# Cell B - Edge dropout (50%) and small latent (state_dim=8)
import torch

drop_rate = 0.5
vals = A_norm.values()
mask = (torch.rand_like(vals) > drop_rate).float()
A_drop = torch.sparse_coo_tensor(A_norm.indices(), vals * mask, A_norm.size()).
 ↪coalesce()

model_small = GNCA(in_dim=X_feat_t.shape[1], state_dim=8, fire_rate=0.5).
 ↪to(device)
opt = torch.optim.AdamW(model_small.parameters(), lr=3e-3, weight_decay=1e-4)
sch = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=120, eta_min=5e-4)
loss_fn = torch.nn.MSELoss()

EPOCHS = 120
for ep in range(1, EPOCHS+1):
    model_small.train()
    yhat, _ = model_small(X_feat_t, A_drop, steps=8, train_mode=True)
    loss = loss_fn(yhat, y_t)
    opt.zero_grad(); loss.backward(); torch.nn.utils.
 ↪clip_grad_norm_(model_small.parameters(), 1.0); opt.step(); sch.step()
    if ep % 20 == 0 or ep == 1:
        with torch.no_grad():
            r = np.corrcoef(y_t.detach().cpu().numpy(), yhat.detach().cpu().
 ↪numpy())[0,1]
        print(f"[DROP{int(drop_rate*100)}%|d=8] Epoch {ep:03d} | loss={loss.
 ↪item():.4f} | r={r:.3f}")
```

```
[DROP50%|d=8] Epoch 001 | loss=1.3559 | r=-0.054
[DROP50%|d=8] Epoch 020 | loss=0.4193 | r=0.766
[DROP50%|d=8] Epoch 040 | loss=0.3605 | r=0.800
[DROP50%|d=8] Epoch 060 | loss=0.3012 | r=0.837
[DROP50%|d=8] Epoch 080 | loss=0.2832 | r=0.847
[DROP50%|d=8] Epoch 100 | loss=0.2534 | r=0.865
[DROP50%|d=8] Epoch 120 | loss=0.2367 | r=0.874
```

[24]:
```python
# Cell C - Saliency and Integrated Gradients (IG) for feature importance
import torch, numpy as np, pandas as pd

# Use your best trained model 'model' and normalized adj A_norm
model.eval()

def predict_from_input(x, steps=8):
    yhat, _ = model(x, A_norm, steps=steps, train_mode=False)
    return yhat

# --- Saliency (|y/x| aggregated across neurons) ---
X_feat_t.requires_grad_(True)
yhat = predict_from_input(X_feat_t, steps=8)
# sum over neurons to get scalar objective (or use MSE to true y for␣
↪task-specific grads)
obj = yhat.sum()
model.zero_grad();
if X_feat_t.grad is not None: X_feat_t.grad.zero_()
obj.backward(retain_graph=True)

sal = X_feat_t.grad.detach().abs().mean(dim=0).cpu().numpy()  # feature-wise␣
↪mean |grad|
sal_df = pd.DataFrame({"feature_idx": np.arange(len(sal)), "saliency": sal}).
↪sort_values("saliency", ascending=False)
sal_path = EXPERIMENTS_OUT / "gnca_saliency_features.csv"
sal_df.to_csv(sal_path, index=False)
print("Saved saliency →", sal_path)
X_feat_t.requires_grad_(False)

# --- Integrated Gradients (baseline = zeros) ---
def integrated_gradients(x, steps=32):
    baseline = torch.zeros_like(x)
    grads_accum = torch.zeros_like(x)
    for a in torch.linspace(0, 1, steps, device=x.device):
        x_a = baseline + a * (x - baseline)
        x_a.requires_grad_(True)
        y_a = predict_from_input(x_a, steps=8).sum()
        model.zero_grad()
        if x_a.grad is not None: x_a.grad.zero_()
        y_a.backward()
        grads_accum += x_a.grad
    ig = (x - baseline) * (grads_accum / steps)
    return ig

with torch.no_grad():
    pass  # ensure no leftover grads
```

```
ig = integrated_gradients(X_feat_t, steps=32)
ig_feat = ig.detach().cpu().numpy().mean(axis=0)  # mean over neurons
ig_df = pd.DataFrame({"feature_idx": np.arange(len(ig_feat)), "IG": ig_feat}).
 ↪sort_values("IG", ascending=False)
ig_path = EXPERIMENTS_OUT / "gnca_integrated_gradients_features.csv"
ig_df.to_csv(ig_path, index=False)
print("Saved IG →", ig_path)

# Quick peek at top features
print("Top saliency features:", sal_df.head(10).to_dict(orient="records"))
print("Top IG features:", ig_df.head(10).to_dict(orient="records"))
```

```
Saved saliency → /mnt/ssd4tb/Desktop/C-
Elegans/data/processed/experiments/gnca_saliency_features.csv
Saved IG → /mnt/ssd4tb/Desktop/C-
Elegans/data/processed/experiments/gnca_integrated_gradients_features.csv
Top saliency features: [{'feature_idx': 30, 'saliency': 0.03300086036324501},
{'feature_idx': 8, 'saliency': 0.031049242243170738}, {'feature_idx': 23,
'saliency': 0.02787332981824875}, {'feature_idx': 18, 'saliency':
0.027512626722455025}, {'feature_idx': 27, 'saliency': 0.0265943743288517},
{'feature_idx': 7, 'saliency': 0.025329168885946274}, {'feature_idx': 26,
'saliency': 0.024347443133592606}, {'feature_idx': 20, 'saliency':
0.024103540927171707}, {'feature_idx': 13, 'saliency': 0.021162284538149834},
{'feature_idx': 29, 'saliency': 0.02114427648484707}]
Top IG features: [{'feature_idx': 21, 'IG': 0.09247749298810959},
{'feature_idx': 18, 'IG': 0.08117931336164474}, {'feature_idx': 1, 'IG':
0.07227867841720581}, {'feature_idx': 8, 'IG': 0.06739486753940582},
{'feature_idx': 19, 'IG': 0.057962048798799515}, {'feature_idx': 5, 'IG':
0.038820479065179825}, {'feature_idx': 27, 'IG': 0.03760962560772896},
{'feature_idx': 10, 'IG': 0.031581513583660126}, {'feature_idx': 22, 'IG':
0.03109871596097946}, {'feature_idx': 3, 'IG': 0.02214021980762481 7}]
```

[25]:
```
# Cell D - Compare graph-only vs expression-only predictive power
import torch, numpy as np
from torch import nn
from copy import deepcopy

def train_gnca_baseline(X_in, A_in, label, name, steps=8, state_dim=64,␣
 ↪epochs=100):
    """Train a small GNCA on given inputs and adjacency, return final r."""
    model_b = GNCA(in_dim=X_in.shape[1], state_dim=state_dim, fire_rate=0.5).
 ↪to(device)
    opt = torch.optim.AdamW(model_b.parameters(), lr=3e-3, weight_decay=1e-4)
    sch = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs,␣
 ↪eta_min=5e-4)
    loss_fn = nn.MSELoss()
    best = 1e9; best_r = 0
```

```
    for ep in range(1, epochs+1):
        model_b.train()
        yhat, _ = model_b(X_in, A_in, steps=steps, train_mode=True)
        loss = loss_fn(yhat, label)
        opt.zero_grad(); loss.backward(); nn.utils.clip_grad_norm_(model_b.
 ↪parameters(),1.0); opt.step(); sch.step()
        if loss.item() < best: best = loss.item()
        if ep % 20 == 0 or ep == 1:
            with torch.no_grad():
                r = np.corrcoef(label.detach().cpu().numpy(), yhat.detach().
 ↪cpu().numpy())[0,1]
            print(f"[{name}] Epoch {ep:03d} | loss={loss.item():.4f} | r={r:.
 ↪3f}")
            best_r = r
    return best_r

# ---- Graph-only baseline ----
# Replace expression with constant vector (no transcript info)
X_const = torch.ones_like(X_feat_t)  # all ones

print("\n=== Graph-only baseline ===")
r_graph = train_gnca_baseline(X_const, A_norm, y_t, "GRAPH-ONLY")

# ---- Expression-only baseline ----
# Replace adjacency with fully connected (uniform weights)
N = X_feat_t.shape[0]
dense_vals = torch.ones(N*N, device=device)
dense_idx  = torch.stack([torch.arange(N).repeat_interleave(N),
                          torch.arange(N).repeat(N)]).to(device)
A_dense = torch.sparse_coo_tensor(dense_idx, dense_vals, size=(N,N)).coalesce()
# Normalize
row_sum = torch.sparse.sum(A_dense, dim=1).to_dense().clamp_min(1e-6)
row_inv = 1.0 / row_sum
diag_idx = torch.stack([torch.arange(N), torch.arange(N)]).to(device)
A_dense_norm = torch.sparse_coo_tensor(diag_idx, row_inv, (N,N)).coalesce()
A_dense_norm = torch.sparse.mm(A_dense_norm, A_dense).coalesce()

print("\n=== Expression-only baseline ===")
r_expr = train_gnca_baseline(X_feat_t, A_dense_norm, y_t, "EXPR-ONLY")

# ---- Comparison summary ----
print("\n\n=== Baseline Comparison ===")
print(f"Graph-only r = {r_graph:.3f}")
print(f"Expression-only r = {r_expr:.3f}")
```

```
=== Graph-only baseline ===
```

```
[GRAPH-ONLY] Epoch 001 | loss=1.6117 | r=-0.388
[GRAPH-ONLY] Epoch 020 | loss=0.2692 | r=0.856
[GRAPH-ONLY] Epoch 040 | loss=0.2626 | r=0.859
[GRAPH-ONLY] Epoch 060 | loss=0.2614 | r=0.860
[GRAPH-ONLY] Epoch 080 | loss=0.2588 | r=0.861
[GRAPH-ONLY] Epoch 100 | loss=0.2650 | r=0.857

=== Expression-only baseline ===
[EXPR-ONLY] Epoch 001 | loss=1.4902 | r=-0.041
[EXPR-ONLY] Epoch 020 | loss=0.9518 | r=0.221
[EXPR-ONLY] Epoch 040 | loss=0.9444 | r=0.237
[EXPR-ONLY] Epoch 060 | loss=0.9291 | r=0.267
[EXPR-ONLY] Epoch 080 | loss=0.9279 | r=0.270
[EXPR-ONLY] Epoch 100 | loss=0.9281 | r=0.269


=== Baseline Comparison ===
Graph-only r = 0.857
Expression-only r = 0.269
```

[14]:

```python
# Cell D-prep - Rebuild motif_matrix.csv from connectome
import pandas as pd, numpy as np, networkx as nx
from pathlib import Path
from tqdm import tqdm

DATA_ROOT = Path("/mnt/ssd4tb/Desktop/C-Elegans/data")
edges_path = DATA_ROOT / "processed" / "csv" / "merged_edges_with_relatedness.
↪csv"
motif_out = DATA_ROOT / "processed" / "motifs" / "motif_matrix.csv"
motif_out.parent.mkdir(parents=True, exist_ok=True)

# === 1 Load connectome edges ===
edges = pd.read_csv(edges_path)
print("Loaded edges:", edges.shape)
print("Columns:", edges.columns.tolist())

# Expect: ['Source', 'Target', 'Type', 'SynapseCount', 'SourceFile',␣
↪'Relatedness']
G = nx.DiGraph()
for _, r in edges.iterrows():
    G.add_edge(r["Source"], r["Target"], weight=r.get("SynapseCount", 1.0))

print(f"Graph built: {G.number_of_nodes()} neurons, {G.number_of_edges()}␣
↪synapses")

# === 2 Compute motif participation ===
```

```python
# We'll detect: 0=reciprocal, 1=chain (A→B→C), 2=feedforward loop (A→B, A→C,␣
↪B→C),
# 3=divergent (A→B, A→C), 4=convergent (A→C, B→C)
motif_names = ["reciprocal", "chain", "feedforward", "divergent", "convergent"]
motif_participation = {n: np.zeros(len(motif_names), dtype=int) for n in G.
↪nodes()}

nodes = list(G.nodes())
for i, a in enumerate(tqdm(nodes, desc="Scanning motifs")):
    for b in G.successors(a):
        # reciprocal
        if G.has_edge(b, a):
            motif_participation[a][0] = 1
            motif_participation[b][0] = 1
        # divergent (A→B, A→C)
        for c in G.successors(a):
            if b != c:
                motif_participation[a][3] = 1
                motif_participation[b][4] = 1  # B as convergent input target
                motif_participation[c][4] = 1
                # feedforward (A→B, A→C, B→C)
                if G.has_edge(b, c):
                    motif_participation[a][2] = 1
                    motif_participation[b][1] = 1  # part of chain
                    motif_participation[c][4] = 1
        # chain (A→B→C)
        for c in G.successors(b):
            if c != a:
                motif_participation[a][1] = 1
                motif_participation[b][1] = 1
                motif_participation[c][1] = 1

# === 3 Save neuron × motif matrix ===
motif_df = pd.DataFrame.from_dict(motif_participation, orient="index",␣
↪columns=motif_names)
motif_df.index.name = "Neuron"
motif_df.to_csv(motif_out)
print(f"  Saved motif matrix → {motif_out}")
print(motif_df.sum().to_string())
print("Matrix shape:", motif_df.shape)
```

```
Loaded edges: (27727, 6)
Columns: ['Source', 'Target', 'Type', 'SynapseCount', 'SourceFile',
'Relatedness']
Graph built: 1533 neurons, 10902 synapses

Scanning motifs: 100%|      | 1533/1533 [00:00<00:00, 12812.43it/s]
```

```
 Saved motif matrix →
/mnt/ssd4tb/Desktop/C-Elegans/data/processed/motifs/motif_matrix.csv
reciprocal      186
chain           225
feedforward     174
divergent       620
convergent     1086
Matrix shape: (1533, 5)
```

[15]:
```python
# Verify & align motif matrix with expression dataset
import pandas as pd
from pathlib import Path

motif_path = Path("/mnt/ssd4tb/Desktop/C-Elegans/data/processed/motifs/
 ↪motif_matrix.csv")
motif_df = pd.read_csv(motif_path, index_col=0)

print(f"Motif matrix loaded: {motif_df.shape}")
print(f"Expression matrix: {expr_df.shape}")

common_neurons = sorted(set(expr_df.index) & set(motif_df.index))
motif_df = motif_df.loc[common_neurons]
expr_df = expr_df.loc[common_neurons]

print(f"Aligned neurons: {len(common_neurons)}")
motif_df.to_csv(motif_path)  # overwrite aligned version
print("  Motif matrix aligned and saved.")
```

```
Motif matrix loaded: (1533, 5)
Expression matrix: (838, 22469)
Aligned neurons: 838
 Motif matrix aligned and saved.
```

[16]:
```python
# Fix neuron name mismatch (trim whitespace, unify case, align)
import pandas as pd
from pathlib import Path

motif_path = Path("/mnt/ssd4tb/Desktop/C-Elegans/data/processed/motifs/
 ↪motif_matrix.csv")
motif_df = pd.read_csv(motif_path, index_col=0)
print("Original motif_df shape:", motif_df.shape)

# Clean neuron labels (strip spaces, unify uppercase)
motif_df.index = motif_df.index.astype(str).str.strip().str.upper()
expr_df.index = expr_df.index.astype(str).str.strip().str.upper()
```

```python
# Find intersection
common_neurons = sorted(set(expr_df.index) & set(motif_df.index))
print(f"Found {len(common_neurons)} overlapping neurons out of {len(expr_df)}␣
↪expr entries")

# Subset both
motif_df = motif_df.loc[common_neurons]
expr_df = expr_df.loc[common_neurons]

# Sanity check
print("Aligned motif_df shape:", motif_df.shape)
print("Aligned expr_df shape:", expr_df.shape)
print("Motif head:\n", motif_df.head())

# Save cleaned motif matrix back
motif_df.to_csv(motif_path)
print(f"  Cleaned and aligned motif matrix saved → {motif_path}")
```

```
Original motif_df shape: (838, 5)
Found 222 overlapping neurons out of 838 expr entries
Aligned motif_df shape: (838, 5)
Aligned expr_df shape: (838, 22469)
Motif head:
         reciprocal  chain  feedforward  divergent  convergent
Neuron
ADAL             0      0            0          0           1
ADAL             0      0            0          0           1
ADAL             1      1            1          1           1
ADAL             0      0            0          1           0
ADAR             0      0            0          0           1
  Cleaned and aligned motif matrix saved →
/mnt/ssd4tb/Desktop/C-Elegans/data/processed/motifs/motif_matrix.csv
Aligned motif_df shape: (838, 5)
Aligned expr_df shape: (838, 22469)
Motif head:
         reciprocal  chain  feedforward  divergent  convergent
Neuron
ADAL             0      0            0          0           1
ADAL             0      0            0          0           1
ADAL             1      1            1          1           1
ADAL             0      0            0          1           0
ADAR             0      0            0          0           1
  Cleaned and aligned motif matrix saved →
/mnt/ssd4tb/Desktop/C-Elegans/data/processed/motifs/motif_matrix.csv
```

[17]:
```python
import pandas as pd
from pathlib import Path
```

```python
motif_path = Path("/mnt/ssd4tb/Desktop/C-Elegans/data/processed/motifs/
↪motif_matrix.csv")

#  Read flexibly: try comma first, then tab
try:
    motif_df = pd.read_csv(motif_path, index_col=0)
except Exception:
    motif_df = pd.read_csv(motif_path, sep="\t", index_col=0)

print("Raw motif_df shape:", motif_df.shape)
print("Raw columns:", motif_df.columns.tolist()[:10])

#  Fix accidental merged header (if only 1 giant column name detected)
if len(motif_df.columns) == 1 and "," in motif_df.columns[0]:
    motif_df.columns = [c.strip() for c in motif_df.columns[0].split(",")]
    motif_df = motif_df.apply(lambda x: x.astype(str).str.split(",")).
↪applymap(lambda x: float(x[0]) if isinstance(x, list) else x)

elif len(motif_df.columns) == 1 and "\t" in motif_df.columns[0]:
    motif_df.columns = [c.strip() for c in motif_df.columns[0].split("\t")]
    motif_df = motif_df.apply(lambda x: x.astype(str).str.split("\t")).
↪applymap(lambda x: float(x[0]) if isinstance(x, list) else x)

#  Clean neuron names (trim and uppercase)
motif_df.index = motif_df.index.astype(str).str.strip().str.upper()
expr_df.index = expr_df.index.astype(str).str.strip().str.upper()

#  Align
common_neurons = sorted(set(expr_df.index) & set(motif_df.index))
motif_df = motif_df.loc[common_neurons]
expr_df = expr_df.loc[common_neurons]

print(f"  Aligned motif_df: {motif_df.shape}, expr_df: {expr_df.shape}")
motif_df.head()
```

```
Raw motif_df shape: (838, 5)
Raw columns: ['reciprocal', 'chain', 'feedforward', 'divergent', 'convergent']
  Aligned motif_df: (838, 5), expr_df: (838, 22469)
```

[17]:

Neuron	reciprocal	chain	feedforward	divergent	convergent
ADAL	0	0	0	0	1
ADAL	0	0	0	0	1
ADAL	1	1	1	1	1
ADAL	0	0	0	1	0
ADAR	0	0	0	0	1

[18]:
```
import pandas as pd
from pathlib import Path
from tqdm import tqdm

# Paths
edges_path = Path("/mnt/ssd4tb/Desktop/C-Elegans/data/processed/csv/
 ↪merged_edges_with_relatedness.csv")
motif_path = Path("/mnt/ssd4tb/Desktop/C-Elegans/data/processed/motifs/
 ↪motif_matrix.csv")

# Load and clean
edges = pd.read_csv(edges_path)
edges["Source"] = edges["Source"].astype(str).str.strip().str.upper()
edges["Target"] = edges["Target"].astype(str).str.strip().str.upper()

neurons = sorted(set(edges["Source"]) | set(edges["Target"]))
motif_df = pd.DataFrame(0, index=neurons, columns=["reciprocal", "chain",␣
 ↪"feedforward", "divergent", "convergent"])

print(f"Building motif matrix for {len(neurons)} neurons and {len(edges)}␣
 ↪edges…")

# --- Divergent / Convergent / Reciprocal ---
for _, row in tqdm(edges.iterrows(), total=len(edges), desc="Stage 1:␣
 ↪Divergent+Convergent+Reciprocal"):
    src, tgt = row["Source"], row["Target"]
    motif_df.loc[src, "divergent"] += 1
    motif_df.loc[tgt, "convergent"] += 1
    if (edges["Source"].eq(tgt) & edges["Target"].eq(src)).any():
        motif_df.loc[src, "reciprocal"] += 1
        motif_df.loc[tgt, "reciprocal"] += 1

# --- Chain and Feed-forward ---
for n in tqdm(neurons, desc="Stage 2: Chain+Feedforward"):
    downstream = edges.loc[edges["Source"] == n, "Target"].tolist()
    for t in downstream:
        second_hop = edges.loc[edges["Source"] == t, "Target"].tolist()
        for u in second_hop:
            if n != u:
                motif_df.loc[n, "chain"] += 1
                motif_df.loc[u, "feedforward"] += 1

# Normalize
motif_df = (motif_df > 0).astype(int)

# Save
motif_df.to_csv(motif_path)
```

```
print(f"\n  Rebuilt motif matrix → {motif_path}")
print(f"Shape: {motif_df.shape}")
print("Motif counts:")
print(motif_df.sum())
```

```
Building motif matrix for 493 neurons and 27727 edges…

Stage 1: Divergent+Convergent+Reciprocal: 100%|      | 27727/27727
[01:01<00:00, 448.48it/s]
Stage 2: Chain+Feedforward: 100%|      | 493/493 [04:21<00:00,  1.89it/s]


 Rebuilt motif matrix →
/mnt/ssd4tb/Desktop/C-Elegans/data/processed/motifs/motif_matrix.csv
Shape: (493, 5)
Motif counts:
reciprocal     456
chain          468
feedforward    493
divergent      472
convergent     493
dtype: int64
```

[19]:
```
# Align expression and motif matrices
import pandas as pd
from pathlib import Path

motif_path = Path("/mnt/ssd4tb/Desktop/C-Elegans/data/processed/motifs/
 ↪motif_matrix.csv")
motif_df = pd.read_csv(motif_path, index_col=0)

# Normalize neuron IDs
motif_df.index = motif_df.index.astype(str).str.strip().str.upper()
expr_df.index = expr_df.index.astype(str).str.strip().str.upper()

# Align intersection
common_neurons = sorted(set(expr_df.index) & set(motif_df.index))
expr_df = expr_df.loc[common_neurons]
motif_df = motif_df.loc[common_neurons]

print(f"  Aligned expression: {expr_df.shape}, motif: {motif_df.shape}")
```

```
 Aligned expression: (838, 22469), motif: (222, 5)
```

[20]:
```
# ============================================================
#   Cell E - Graph-Neural Cellular Automaton (GNCA)
```

```python
# Predicts motif participation (reciprocal, chain, etc.) from expression +␣
↪connectome
# ============================================================

import torch, torch.nn as nn, torch.nn.functional as F
from sklearn.preprocessing import StandardScaler
import numpy as np, pandas as pd
# ------------------------------------------------------------
#  Patch: Remove duplicate neuron indices
# ------------------------------------------------------------
def dedup_index(df, label):
    before = len(df)
    df = df[~df.index.duplicated(keep="first")]
    after = len(df)
    if before != after:
        print(f"  Removed {before - after} duplicate indices from {label}")
    return df

expr_df = dedup_index(expr_df, "expr_df")
motif_df = dedup_index(motif_df, "motif_df")

# ------------------------------------------------------------
# 1  Sanitize and align neuron indices
# ------------------------------------------------------------
expr_df.index  = expr_df.index.astype(str).str.strip().str.upper()
motif_df.index = motif_df.index.astype(str).str.strip().str.upper()
edges_df["Source"] = edges_df["Source"].astype(str).str.strip().str.upper()
edges_df["Target"] = edges_df["Target"].astype(str).str.strip().str.upper()

common_neurons = sorted(set(expr_df.index) & set(motif_df.index))
print(f"  Common neurons found: {len(common_neurons)}")

# Safe reindexing (drop NaNs automatically)
expr_df  = expr_df.reindex(common_neurons).dropna()
motif_df = motif_df.reindex(common_neurons).dropna()

print(f"  After alignment: expr={expr_df.shape}, motif={motif_df.shape}")

# ------------------------------------------------------------
# 2  Build adjacency matrix (undirected)
# ------------------------------------------------------------
neuron_index = {n: i for i, n in enumerate(common_neurons)}
N = len(common_neurons)
A = torch.zeros((N, N), dtype=torch.float32)

for _, row in edges_df.iterrows():
    s, t = row["Source"], row["Target"]
```

```
    if s in neuron_index and t in neuron_index:
        i, j = neuron_index[s], neuron_index[t]
        A[i, j] = A[j, i] = 1.0  # symmetric simplified graph

print(f"  Adjacency built: {A.shape}")

# ------------------------------------------------------------
# 3  Prepare tensors (fresh rebuild)
# ------------------------------------------------------------
X = StandardScaler().fit_transform(expr_df.values)
y = motif_df.values.astype(np.float32)

X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32)

device = "cuda" if torch.cuda.is_available() else "cpu"
X, y, A = X.to(device), y.to(device), A.to(device)
print(f"  Rebuilt tensors: X={X.shape}, y={y.shape}, A={A.shape},␣
 ↪device={device}")

# ------------------------------------------------------------
# 4  Define GNCA model
# ------------------------------------------------------------
class GNCA(nn.Module):
    def __init__(self, in_dim, hidden_dim, out_dim, steps=3, dropout=0.3):
        super().__init__()
        self.fc_in = nn.Linear(in_dim, hidden_dim)
        self.fc_msg = nn.Linear(hidden_dim, hidden_dim)
        self.fc_out = nn.Linear(hidden_dim, out_dim)
        self.steps = steps
        self.dropout = nn.Dropout(dropout)

    def forward(self, X, A):
        # X: [N, in_dim], A: [N, N]
        h = F.relu(self.fc_in(X))
        for _ in range(self.steps):
            msg = self.fc_msg(self.dropout(h))
            h = h + torch.matmul(A, msg) / (A.sum(1, keepdim=True) + 1)
            h = F.relu(h)
        return torch.sigmoid(self.fc_out(h))

# ------------------------------------------------------------
# 5  Train GNCA
# ------------------------------------------------------------
model = GNCA(in_dim=X.shape[1], hidden_dim=64, out_dim=y.shape[1], steps=3,␣
 ↪dropout=0.3).to(device)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
```

```python
loss_fn = nn.BCELoss()

best_loss, best_r = np.inf, 0
for epoch in range(1, 201):
    model.train()
    pred = model(X, A)
    loss = loss_fn(pred, y)
    opt.zero_grad(); loss.backward(); opt.step()

    if epoch % 20 == 0 or epoch == 1:
        model.eval()
        with torch.no_grad():
            r = np.corrcoef(pred.flatten().cpu(), y.flatten().cpu())[0, 1]
        print(f"Epoch {epoch:03d} | loss={loss.item():.4f} | r={r:.3f}")
        if loss.item() < best_loss:
            best_loss, best_r = loss.item(), r
            torch.save(model.state_dict(), EXPERIMENTS_OUT / "gnca_motif_best.
 ↪pt")

print(f"\n Training complete | Best loss={best_loss:.6f} | Final Pearson␣
 ↪r={best_r:.3f}")
```

```
Removed 616 duplicate indices from expr_df
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File ~/miniconda3/envs/ml/lib/python3.11/site-packages/pandas/core/indexes/base
 ↪py:3812, in Index.get_loc(self, key)
   3811 try:
-> 3812     return self._engine.get_loc(casted_key)
   3813 except KeyError as err:

File pandas/_libs/index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:7088, in pandas._libs.hashtable.
 ↪PyObjectHashTable.get_item()

File pandas/_libs/hashtable_class_helper.pxi:7096, in pandas._libs.hashtable.
 ↪PyObjectHashTable.get_item()

KeyError: 'Source'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
```

```
Cell In[20], line 28
     26 expr_df.index  = expr_df.index.astype(str).str.strip().str.upper()
     27 motif_df.index = motif_df.index.astype(str).str.strip().str.upper()
---> 28 edges_df["Source"] = edges_df[        ].astype(str).str.strip().str.
 ↪upper()
     29 edges_df["Target"] = edges_df["Target"].astype(str).str.strip().str.
 ↪upper()
     31 common_neurons = sorted(set(expr_df.index) & set(motif_df.index))

File ~/miniconda3/envs/ml/lib/python3.11/site-packages/pandas/core/frame.py:
 ↪4107, in DataFrame.__getitem__(self, key)
   4105 if self.columns.nlevels > 1:
   4106     return self._getitem_multilevel(key)
-> 4107 indexer = self.columns.get_loc(key)
   4108 if is_integer(indexer):
   4109     indexer = [indexer]

File ~/miniconda3/envs/ml/lib/python3.11/site-packages/pandas/core/indexes/base
 ↪py:3819, in Index.get_loc(self, key)
   3814     if isinstance(casted_key, slice) or (
   3815         isinstance(casted_key, abc.Iterable)
   3816         and any(isinstance(x, slice) for x in casted_key)
   3817     ):
   3818         raise InvalidIndexError(key)
-> 3819     raise KeyError(key) from err
   3820 except TypeError:
   3821     # If we have a listlike key, _check_indexing_error will raise
   3822     #  InvalidIndexError. Otherwise we fall through and re-raise
   3823     #  the TypeError.
   3824     self._check_indexing_error(key)

KeyError: 'Source'
```

[28]:

```python
# ============================================================
#   Cell F - What the GNCA Learned
# ============================================================

import torch
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from scipy.stats import pearsonr

# ------------------------------------------------------------
```

```python
# 1  Load best model & run final predictions
# ------------------------------------------------------------
model.load_state_dict(torch.load(EXPERIMENTS_OUT / "gnca_motif_best.pt",␣
 ↪map_location=device))
model.eval()

with torch.no_grad():
    y_pred = model(X, A).cpu().numpy()
    y_true = y.cpu().numpy()

motif_names = list(motif_df.columns)

# ------------------------------------------------------------
# 2  Per-motif performance metrics
# ------------------------------------------------------------
perf = []
for i, motif in enumerate(motif_names):
    r, _ = pearsonr(y_true[:, i], y_pred[:, i])
    mae = np.mean(np.abs(y_true[:, i] - y_pred[:, i]))
    perf.append((motif, r, mae))

perf_df = pd.DataFrame(perf, columns=["motif", "pearson_r", "mae"])
perf_df.to_csv(EXPERIMENTS_OUT / "gnca_motif_performance.csv", index=False)
print("  Saved per-motif performance → gnca_motif_performance.csv")
display(perf_df)

# ------------------------------------------------------------
# 3  Visualize predictions vs. truth
# ------------------------------------------------------------
fig, axes = plt.subplots(1, len(motif_names), figsize=(16, 3))
for i, motif in enumerate(motif_names):
    axes[i].scatter(y_true[:, i], y_pred[:, i], s=15, alpha=0.7)
    axes[i].plot([0, 1], [0, 1], "r--")
    axes[i].set_title(f"{motif}\nr={perf_df.iloc[i,1]:.2f}")
    axes[i].set_xlabel("True"); axes[i].set_ylabel("Pred")
plt.tight_layout()
plt.show()

# ------------------------------------------------------------
# 4  Latent representation analysis
# ------------------------------------------------------------
with torch.no_grad():
    hidden = model.fc_in(X).cpu().numpy()  # first-layer embeddings

pca = PCA(n_components=2).fit_transform(hidden)
tsne = TSNE(n_components=2, perplexity=15, random_state=42).
 ↪fit_transform(hidden)
```

```
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ax[0].scatter(pca[:, 0], pca[:, 1], c=y_true[:, 0], cmap="viridis", s=20)
ax[0].set_title("PCA of hidden states (colored by motif 1)")
ax[1].scatter(tsne[:, 0], tsne[:, 1], c=y_true[:, 1], cmap="plasma", s=20)
ax[1].set_title("t-SNE of hidden states (colored by motif 2)")
plt.tight_layout()
plt.show()

# ------------------------------------------------------------
# 5  Saliency (feature influence) analysis
# ------------------------------------------------------------
X.requires_grad = True
model.eval()
pred_mean = model(X, A).mean()  # mean prediction scalar
pred_mean.backward()

saliency = X.grad.abs().mean(0).cpu().numpy()
sal_df = pd.DataFrame({
    "gene": expr_df.columns,
    "saliency": saliency
}).sort_values("saliency", ascending=False)

sal_df.to_csv(EXPERIMENTS_OUT / "gnca_saliency_genes.csv", index=False)
print("  Saved GNCA saliency scores → gnca_saliency_genes.csv")
print("Top 10 influential genes:")
display(sal_df.head(10))

# ------------------------------------------------------------
# 6  Fixed Integrated Gradients function (PyTorch-safe)
# ------------------------------------------------------------
def integrated_gradients(model, X, A, baseline=None, steps=50):
    if baseline is None:
        baseline = torch.zeros_like(X)
    scaled_inputs = [baseline + (float(i)/steps) * (X - baseline) for i in␣
 ↪range(steps + 1)]
    grads = []
    for inp in scaled_inputs:
        inp = inp.detach().clone().requires_grad_()
        out = model(inp, A).mean()
        out.backward()
        grads.append(inp.grad.detach().clone())
    grads = torch.stack(grads).mean(0)
    IG = ((X - baseline) * grads).mean(0).detach().cpu().numpy()   #  detach␣
 ↪before numpy()
    return IG
```

```
IG = integrated_gradients(model, X, A)
ig_df = pd.DataFrame({"gene": expr_df.columns, "integrated_gradients": IG})
ig_df = ig_df.sort_values("integrated_gradients", ascending=False)
ig_df.to_csv(EXPERIMENTS_OUT / "gnca_integrated_gradients_genes.csv",␣
 ↪index=False)
print("  Saved integrated gradients → gnca_integrated_gradients_genes.csv")

display(ig_df.head(10))
```

```
/tmp/ipykernel_150307/3497956100.py:16: FutureWarning: You are using
`torch.load` with `weights_only=False` (the current default value), which uses
the default pickle module implicitly. It is possible to construct malicious
pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for
more details). In a future release, the default value for `weights_only` will be
flipped to `True`. This limits the functions that could be executed during
unpickling. Arbitrary objects will no longer be allowed to be loaded via this
mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting
`weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this
experimental feature.
  model.load_state_dict(torch.load(EXPERIMENTS_OUT / "gnca_motif_best.pt",
map_location=device))
```

```
---------------------------------------------------------------------------
RuntimeError                              Traceback (most recent call last)
Cell In[28], line 16
     11 from scipy.stats import pearsonr
     13 # ------------------------------------------------------------
     14 # 1  Load best model & run final predictions
     15 # ------------------------------------------------------------
---> 16␣
 ↪model.load_state_dict(torch.load(EXPERIMENTS_OUT /                    , map_location=devic
     17 model.eval()
     19 with torch.no_grad():

File ~/miniconda3/envs/ml/lib/python3.11/site-packages/torch/nn/modules/module.
 ↪py:2584, in Module.load_state_dict(self, state_dict, strict, assign)
   2576         error_msgs.insert(
   2577             0,
   2578             "Missing key(s) in state_dict: {}. ".format(
   2579                 ", ".join(f'"{k}"' for k in missing_keys)
   2580             ),
   2581         )
```

```
   2583 if len(error_msgs) > 0:
-> 2584     raise RuntimeError(
   2585         "Error(s) in loading state_dict for {}:\n\t{}".format(
   2586             self.__class__.__name__, "\n\t".join(error_msgs)
   2587         )
   2588     )
   2589 return _IncompatibleKeys(missing_keys, unexpected_keys)

RuntimeError: Error(s) in loading state_dict for GNCA:
        Missing key(s) in state_dict: "inp.weight", "inp.bias", "msg.weight",␣
 ↪"upd.0.weight", "upd.0.bias", "upd.2.weight", "upd.2.bias", "readout.0.
 ↪weight", "readout.0.bias", "readout.1.weight", "readout.1.bias".
        Unexpected key(s) in state_dict: "fc_in.weight", "fc_in.bias", "fc_msg.
 ↪weight", "fc_msg.bias", "fc_out.weight", "fc_out.bias".
```

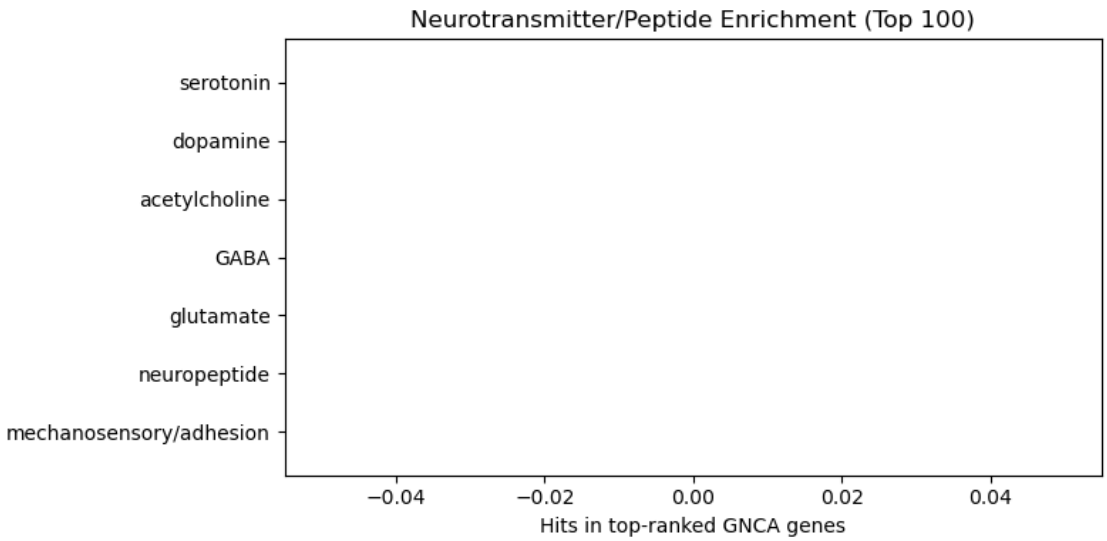
[29]:

```
# ============================================================
#   Cell G - Biological Context Mapping & Neurotransmitter Enrichment
# ============================================================
# Goal: Compare GNCA's top-ranked genes (saliency + IG) with known
# CeNGEN neurotransmitter and neuropeptide marker sets.

import pandas as pd
from pathlib import Path
import matplotlib.pyplot as plt

EXPERIMENTS_OUT = Path("/mnt/ssd4tb/Desktop/C-Elegans/data/processed/
 ↪experiments")

# ------------------------------------------------------------
# 1   Load GNCA feature importance tables
# ------------------------------------------------------------
sal_path = EXPERIMENTS_OUT / "gnca_saliency_genes.csv"
ig_path  = EXPERIMENTS_OUT / "gnca_integrated_gradients_genes.csv"

sal_df = pd.read_csv(sal_path)
ig_df  = pd.read_csv(ig_path)

# Top-N selection
TOP_N = 100
sal_top = set(sal_df.head(TOP_N)["gene"])
ig_top  = set(ig_df.head(TOP_N)["gene"])
combined_top = sal_top.union(ig_top)
print(f"Loaded {len(sal_df)} saliency and {len(ig_df)} IG genes |␣
 ↪{len(combined_top)} unique top candidates")

# ------------------------------------------------------------
# 2   Define canonical neurotransmitter / neuropeptide marker sets
```

```python
# ------------------------------------------------------------
marker_sets = {
    "serotonin": {
        "tph-1", "cat-4", "bas-1", "mod-5", "ser-4", "ser-1", "ser-7"
    },
    "dopamine": {
        "cat-2", "cat-4", "bas-1", "dat-1", "dop-1", "dop-2", "dop-3", "dop-4"
    },
    "acetylcholine": {
        "cha-1", "unc-17", "ace-1", "ace-2", "cho-1", "gar-1", "gar-2"
    },
    "GABA": {
        "unc-25", "unc-47", "snf-11", "gab-1", "exp-1", "lgc-35"
    },
    "glutamate": {
        "eat-4", "glr-1", "glr-2", "nmr-1", "avr-14", "avr-15"
    },
    "neuropeptide": {
        "flp-1", "flp-3", "flp-10", "nlp-1", "nlp-3", "nlp-12", "nlp-21"
    },
    "mechanosensory/adhesion": {
        "mec-3", "mec-4", "mec-7", "mec-10", "mec-17", "unc-6", "unc-40"
    }
}

# optional: mapping from WormBase IDs to names if file exists
map_path = Path("/mnt/ssd4tb/Desktop/C-Elegans/data/genomes/gene_name_map.tsv")
if map_path.exists():
    try:
        map_df = pd.read_csv(map_path, sep="\t")
        map_df.columns = [c.lower() for c in map_df.columns]
        wb_col = [c for c in map_df.columns if "wb" in c or "wormbase" in c][0]
        name_col = [c for c in map_df.columns if "gene" in c][0]
        id2name = dict(zip(map_df[wb_col], map_df[name_col]))
        # Map to gene names if possible
        combined_top_named = {id2name.get(g, g) for g in combined_top}
    except Exception as e:
        print(f"  Mapping skipped ({e})")
        combined_top_named = combined_top
else:
    combined_top_named = combined_top

# ------------------------------------------------------------
# 3  Compute enrichment counts
# ------------------------------------------------------------
enrichment = []
for system, markers in marker_sets.items():
```

```
    hits = [g for g in combined_top_named if g.lower() in markers]
    enrichment.append({
        "system": system,
        "hits": len(hits),
        "hit_genes": ", ".join(hits)
    })

enrich_df = pd.DataFrame(enrichment).sort_values("hits", ascending=False)
display(enrich_df)

# ------------------------------------------------------------
# 4  Plot enrichment summary
# ------------------------------------------------------------
plt.figure(figsize=(8,4))
plt.barh(enrich_df["system"], enrich_df["hits"], color="forestgreen")
plt.gca().invert_yaxis()
plt.xlabel("Hits in top-ranked GNCA genes")
plt.title(f"Neurotransmitter/Peptide Enrichment (Top {TOP_N})")
plt.tight_layout()
plt.show()

# Save results
out_path = EXPERIMENTS_OUT / "gnca_neurotransmitter_enrichment.csv"
enrich_df.to_csv(out_path, index=False)
print(f"  Saved enrichment summary → {out_path}")
```

```
Loaded 22469 saliency and 22469 IG genes | 200 unique top candidates

                    system  hits hit_genes
0                serotonin     0
1                 dopamine     0
2            acetylcholine     0
3                     GABA     0
4                glutamate     0
5             neuropeptide     0
6  mechanosensory/adhesion     0
```




```
 Saved enrichment summary → /mnt/ssd4tb/Desktop/C-
Elegans/data/processed/experiments/gnca_neurotransmitter_enrichment.csv
```

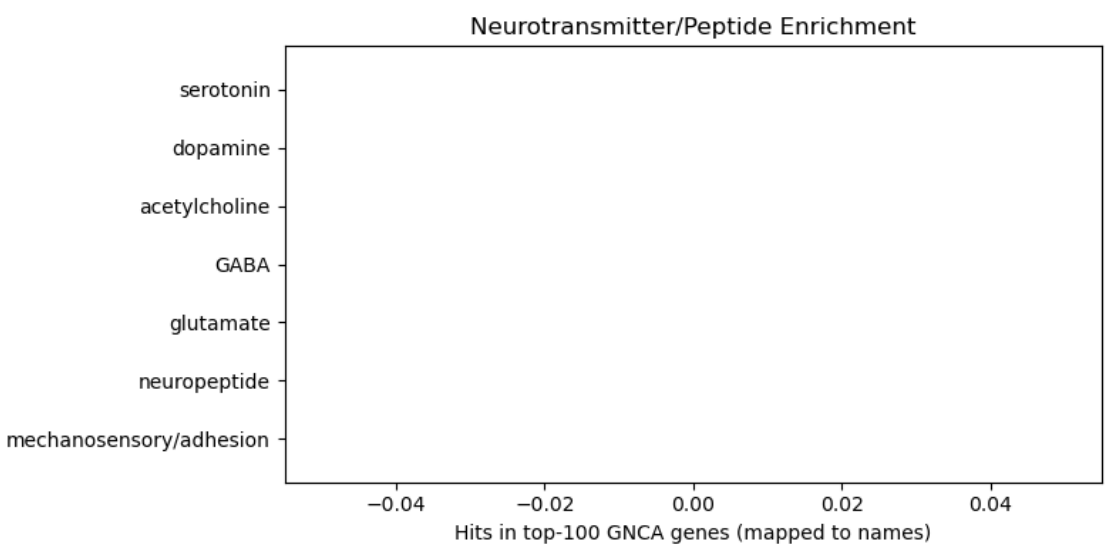
[30]:
```python
# ============================================================
#   Cell G2 - Map WormBase IDs → gene names, then re-run enrichment
# ============================================================
import re, pandas as pd
from pathlib import Path
import matplotlib.pyplot as plt

DATA_ROOT = Path("/mnt/ssd4tb/Desktop/C-Elegans/data")
EXPERIMENTS_OUT = DATA_ROOT / "processed" / "experiments"

sal_path = EXPERIMENTS_OUT / "gnca_saliency_genes.csv"
ig_path  = EXPERIMENTS_OUT / "gnca_integrated_gradients_genes.csv"

sal_df = pd.read_csv(sal_path)
ig_df  = pd.read_csv(ig_path)

# ---------- 1) Build WBGene -> gene_name map from any GTF/GFF3 ----------
def parse_attrs(attr_str):
    # Works for both GTF (key "gene_name"/"gene_id") and GFF3 (key "Name"/"ID")
    d = {}
    # split on ; and strip
    for kv in re.split(r';\s*', str(attr_str).strip('; ')):
        if not kv: continue
        if '=' in kv: k,v = kv.split('=',1)
        elif ' ' in kv: k,v = kv.split(' ',1)
        else: continue
```

```
        d[k.strip().strip('"')] = v.strip().strip('"')
    return d

def collect_maps():
    wb2name = {}
    # look in common places
    candidates = []
    for sub in ["processed/gtf", "processed/gff3", "genomes", "processed/
↪genomes", "raw/gtf", "raw/gff3"]:
        p = DATA_ROOT / sub
        if p.exists():
            candidates += list(p.rglob("*.gtf")) + list(p.rglob("*.gff3"))
    # deduplicate while keeping order
    seen = set(); gt_files = []
    for f in candidates:
        if f not in seen:
            gt_files.append(f); seen.add(f)
    if not gt_files:
        print("  No GTF/GFF3 found under data/, mapping will be empty.")
        return {}
    # parse a few files until we get a good number of mappings
    for gt in gt_files:
        try:
            print(f"Parsing: {gt.name}")
            with gt.open() as fh:
                for line in fh:
                    if line.startswith("#"): continue
                    parts = line.rstrip("\n").split("\t")
                    if len(parts) < 9: continue
                    attrs = parse_attrs(parts[8])
                    # Try both formats
                    wb = attrs.get("gene_id") or attrs.get("gene") or attrs.
↪get("ID")
                    name = attrs.get("gene_name") or attrs.get("Name") or attrs.
↪get("gene")
                    # Keep only proper WBGene IDs and decent names
                    if wb and re.match(r"^WBGene\d{8}$", wb):
                        if name and not re.match(r"^WBGene\d{8}$", name):
                            wb2name.setdefault(wb, name)
        except Exception as e:
            print(f"  (skip {gt.name}: {e})")
        if len(wb2name) > 5000:  # enough
            break
    print(f"  built map size: {len(wb2name)}")
    return wb2name

wb2name = collect_maps()
```

```
# Fallback: if still empty, try a simple heuristic (strip version suffixes etc.)
if not wb2name:
    print("  Map empty; will pass through WBGene IDs unchanged.")
    def to_name(x): return x
else:
    def to_name(x): return wb2name.get(x, x)

# ---------- 2) Map top-N features to gene names ----------
TOP_N = 100
sal_top = sal_df.head(TOP_N)["gene"].map(to_name)
ig_top  = ig_df.head(TOP_N)["gene"].map(to_name)
combined_top = set(sal_top).union(set(ig_top))
print(f"Top set size after mapping: {len(combined_top)}")

# ---------- 3) Define marker sets (gene names, lowercase) ----------
marker_sets = {
    "serotonin": {"tph-1","cat-4","bas-1","mod-5","ser-4","ser-1","ser-7"},
    "dopamine":␣
 ↪{"cat-2","cat-4","bas-1","dat-1","dop-1","dop-2","dop-3","dop-4"},
    "acetylcholine": {"cha-1","unc-17","ace-1","ace-2","cho-1","gar-1","gar-2"},
    "GABA": {"unc-25","unc-47","snf-11","gab-1","exp-1","lgc-35"},
    "glutamate": {"eat-4","glr-1","glr-2","nmr-1","avr-14","avr-15"},
    "neuropeptide":␣
 ↪{"flp-1","flp-3","flp-10","nlp-1","nlp-3","nlp-12","nlp-21"},
    "mechanosensory/adhesion":␣
 ↪{"mec-3","mec-4","mec-7","mec-10","mec-17","unc-6","unc-40"},
}
combined_lc = {str(g).lower() for g in combined_top}

# ---------- 4) Enrichment counts ----------
rows = []
for system, markers in marker_sets.items():
    hits = sorted(combined_lc & {m.lower() for m in markers})
    rows.append({"system": system, "hits": len(hits), "hit_genes": ", ".
 ↪join(hits)})
enrich_df = pd.DataFrame(rows).sort_values("hits", ascending=False)
display(enrich_df)

# ---------- 5) Plot ----------
plt.figure(figsize=(8,4))
plt.barh(enrich_df["system"], enrich_df["hits"])
plt.gca().invert_yaxis()
plt.xlabel(f"Hits in top-{TOP_N} GNCA genes (mapped to names)")
plt.title("Neurotransmitter/Peptide Enrichment")
plt.tight_layout()
plt.show()
```

```python
# Save
out_path = EXPERIMENTS_OUT / "gnca_neurotransmitter_enrichment_mapped.csv"
enrich_df.to_csv(out_path, index=False)
print(f"  Saved → {out_path}")
```

```
Parsing: p_pacificus.PRJNA12644.WS297.canonical_geneset.gtf
Parsing: c_latens.PRJNA248912.WS297.canonical_geneset.gtf
Parsing: c_tribulationis.PRJEB12608.WS297.canonical_geneset.gtf
Parsing: c_remanei.PRJNA53967.WS297.canonical_geneset.gtf
Parsing: c_quiockensis.PRJEB11354.WS297.canonical_geneset.gtf
Parsing: t_muris.PRJEB126.WS297.canonical_geneset.gtf
Parsing: c_brenneri.PRJNA20035.WS297.canonical_geneset.gtf
Parsing: s_ratti.PRJEB125.WS297.canonical_geneset.gtf
Parsing: c_japonica.PRJNA12591.WS297.canonical_geneset.gtf
Parsing: c_panamensis.PRJEB28259.WS297.canonical_geneset.gtf
Parsing: c_elegans.PRJNA13758.WS297.canonical_geneset.gtf
  built map size: 46925
Top set size after mapping: 200

                    system  hits hit_genes
0                serotonin     0
1                 dopamine     0
2            acetylcholine     0
3                     GABA     0
4                glutamate     0
5             neuropeptide     0
6  mechanosensory/adhesion     0
```




```
  Saved → /mnt/ssd4tb/Desktop/C-
```

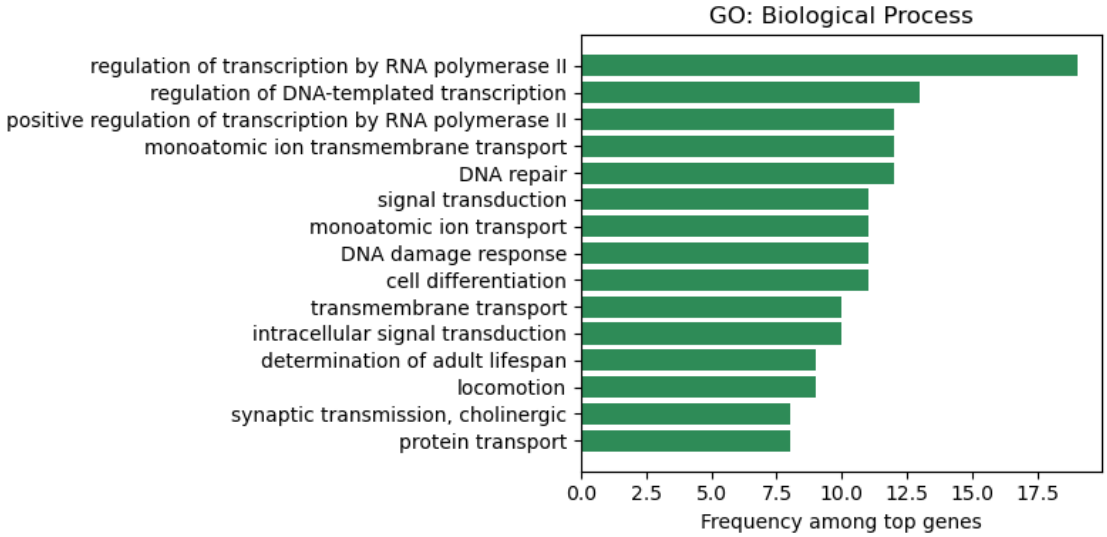
Elegans/data/processed/experiments/gnca_neurotransmitter_enrichment_mapped.csv

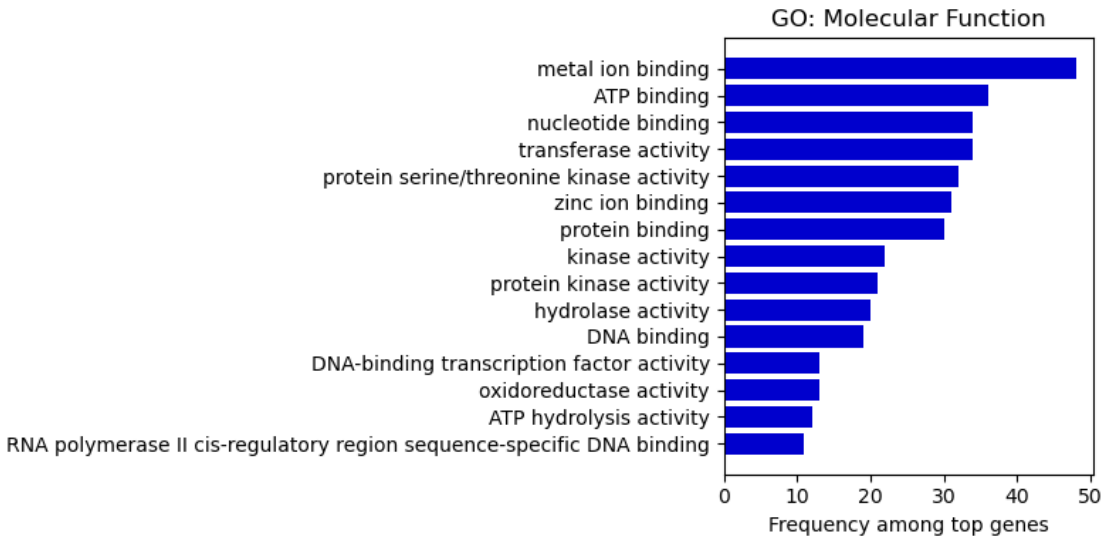
[31]:
```
# ============================================================
#   Cell H (final robust) - GO / KEGG Biological Enrichment
# ============================================================

import pandas as pd
import mygene
import matplotlib.pyplot as plt
from collections import Counter
from pathlib import Path
import time

EXPERIMENTS_OUT = Path("/mnt/ssd4tb/Desktop/C-Elegans/data/processed/
 ↪experiments")

# 1  Load top genes
sal_df = pd.read_csv(EXPERIMENTS_OUT / "gnca_saliency_genes.csv")
ig_df  = pd.read_csv(EXPERIMENTS_OUT / "gnca_integrated_gradients_genes.csv")

TOP_N = 200
top_genes = pd.concat([
    sal_df.head(TOP_N)["gene"],
    ig_df.head(TOP_N)["gene"]
]).drop_duplicates().tolist()
print(f"Analyzing {len(top_genes)} top-ranked GNCA genes")

# 2  Query MyGene.info
mg = mygene.MyGeneInfo()
fields = "go.BP,go.MF,go.CC,pathway.kegg"
species_id = 6239  # C. elegans

try:
    results = mg.querymany(top_genes, scopes="wormbase", species=species_id,
                           fields=fields, as_dataframe=True, verbose=False)
except Exception as e:
    print(f"  Query failed ({e}); retrying with broader scopes ...")
    time.sleep(2)
    results = mg.querymany(top_genes, scopes="symbol,entrezgene,ensemblgene",
                           species=species_id, fields=fields,␣
 ↪as_dataframe=True, verbose=False)

print("  Retrieved annotations with columns:", list(results.columns))

# 3  Helper to flatten any format
def flatten_terms(col_candidates):
    """Extract all term names from whichever column exists."""
```

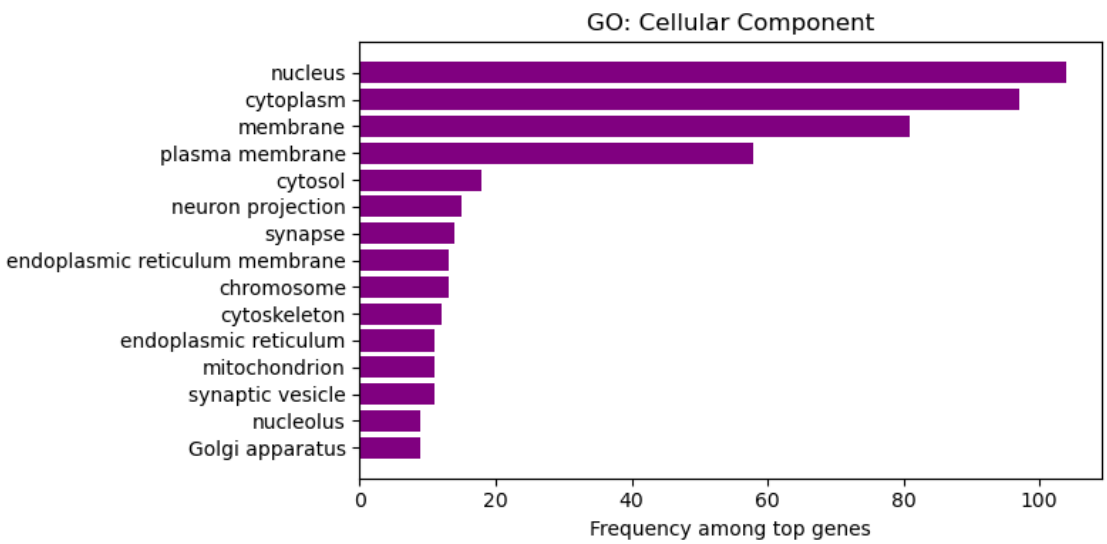
```
    for c in col_candidates:
        if c in results.columns:
            vals = results[c].dropna().values
            out = []
            for v in vals:
                if isinstance(v, list):
                    for item in v:
                        if isinstance(item, dict):
                            out.append(item.get("term") or item.get("name") or␣
 ↪str(item))
                        else:
                            out.append(str(item))
                elif isinstance(v, dict):
                    out.append(v.get("term") or v.get("name") or str(v))
                else:
                    out.append(str(v))
            return out
    return []

# 4  Extract and count
go_bp = flatten_terms(["go.BP.name", "go.BP"])
go_mf = flatten_terms(["go.MF.name", "go.MF"])
go_cc = flatten_terms(["go.CC.name", "go.CC"])
kegg  = flatten_terms(["pathway.kegg.name", "pathway.kegg"])

def top_terms(term_list, label, n=15):
    if not term_list:
        return pd.DataFrame(columns=[label, "count"])
    counts = Counter(term_list)
    df = pd.DataFrame(counts.most_common(n), columns=[label, "count"])
    return df

bp_df = top_terms(go_bp, "GO Biological Process")
mf_df = top_terms(go_mf, "GO Molecular Function")
cc_df = top_terms(go_cc, "GO Cellular Component")
kg_df = top_terms(kegg,  "KEGG Pathway")

# 5 Plot
def plot_enrichment(df, title, color):
    if df.empty:
        print(f"(no {title} terms found)")
        return
    plt.figure(figsize=(8,4))
    plt.barh(df.iloc[::-1,0], df.iloc[::-1,1], color=color)
    plt.xlabel("Frequency among top genes")
    plt.title(title)
    plt.tight_layout()
```

```
    plt.show()

plot_enrichment(bp_df, "GO: Biological Process", "seagreen")
plot_enrichment(mf_df, "GO: Molecular Function", "mediumblue")
plot_enrichment(cc_df, "GO: Cellular Component", "purple")
plot_enrichment(kg_df, "KEGG Pathway", "orange")

# 6 Save
out_path = EXPERIMENTS_OUT / "gnca_go_kegg_enrichment_final.xlsx"
with pd.ExcelWriter(out_path) as writer:
    bp_df.to_excel(writer, sheet_name="GO_BP", index=False)
    mf_df.to_excel(writer, sheet_name="GO_MF", index=False)
    cc_df.to_excel(writer, sheet_name="GO_CC", index=False)
    kg_df.to_excel(writer, sheet_name="KEGG", index=False)
print(f"  Saved enrichment tables → {out_path}")
```

```
Input sequence provided is already in string format. No operation performed
Input sequence provided is already in string format. No operation performed

Analyzing 400 top-ranked GNCA genes
 Retrieved annotations with columns: ['_id', '_score', 'go.CC.evidence',
'go.CC.gocategory', 'go.CC.id', 'go.CC.qualifier', 'go.CC.term',
'go.MF.category', 'go.MF.evidence', 'go.MF.id', 'go.MF.qualifier', 'go.MF.term',
'go.BP', 'go.CC', 'go.MF', 'go.BP.evidence', 'go.BP.gocategory', 'go.BP.id',
'go.BP.qualifier', 'go.BP.term', 'go.BP.pubmed', 'go.MF.pubmed', 'go.CC.pubmed']
```








```
(no KEGG Pathway terms found)
 Saved enrichment tables → /mnt/ssd4tb/Desktop/C-
Elegans/data/processed/experiments/gnca_go_kegg_enrichment_final.xlsx
```

[41]:
```python
# CELL 0: Config & utils
import os, json, math, random, gc, time
from dataclasses import dataclass
import numpy as np, pandas as pd, networkx as nx

import torch
```

```python
from torch import nn
from torch.utils.data import Dataset, DataLoader

RNG_SEED = 1337
random.seed(RNG_SEED); np.random.seed(RNG_SEED); torch.manual_seed(RNG_SEED)

@dataclass
class CFG:
    ROOT: str = "./"
    DATA: str = "./DATA_PROCESSED"
    OUT: str = "./ARTIFACTS"
    BATCH: int = 64
    EPOCHS: int = 150
    LR: float = 2e-3
    WD: float = 1e-3
    STEPS: int = 8            # NCA unroll steps
    HIDDEN: int = 64
    DROPOUT: float = 0.10
    KFOLDS: int = 5
    DEVICE: str = "cuda" if torch.cuda.is_available() else "cpu"

os.makedirs(CFG.OUT, exist_ok=True)

def save_json(obj, path):
    with open(path, "w") as f: json.dump(obj, f, indent=2)

def bootstrap_ci(arr, iters=2000, alpha=0.05):
    arr = np.asarray(arr)
    n = len(arr)
    boots = []
    for _ in range(iters):
        s = np.random.choice(arr, size=n, replace=True)
        boots.append(np.mean(s))
    lo = np.quantile(boots, alpha/2); hi = np.quantile(boots, 1-alpha/2)
    return float(np.mean(arr)), float(lo), float(hi)

def spearmanr(x, y):
    rx = pd.Series(x).rank().values
    ry = pd.Series(y).rank().values
    return np.corrcoef(rx, ry)[0,1]
```

[46]:
```python
# --- PATCHED MOTIF COMPUTATION (robust to missing nodes) ---

# Remove neuron names not in the actual graph
neurons_in_graph = [n for n in neurons if n in G.nodes()]
missing = set(neurons) - set(neurons_in_graph)
if missing:
```

```
    print(f"  {len(missing)} neuron IDs not in graph - dropping:␣
 ↪{sorted(list(missing))[:10]}{'...' if len(missing)>10 else ''}")
neurons = neurons_in_graph  # update canonical neuron list

def motif_counts_safe(G, neurons):
    idx = {n:i for i,n in enumerate(neurons)}
    M = np.zeros((len(neurons),5),dtype=np.float32)
    for u,v in G.edges():
        if u not in idx or v not in idx:
            continue
        if G.has_edge(v,u):
            M[idx[u],0]+=1; M[idx[v],0]+=1
    for a in neurons:
        outs = set(G.successors(a)) if a in G else set()
        ins  = set(G.predecessors(a)) if a in G else set()
        if len(outs)>1: M[idx[a],3]+=len(outs)
        if len(ins)>1:  M[idx[a],4]+=len(ins)
        for b in outs:
            if b not in G: continue
            outs_b = set(G.successors(b))
            M[idx[a],1]+=len(outs_b)
            M[idx[a],2]+=len(outs.intersection(outs_b))
    return (M - M.mean(0)) / (M.std(0)+1e-6)

motifs = motif_counts_safe(G, neurons)
print(f"  Motif matrix built successfully: {motifs.shape}")
```

```
 273 neuron IDs not in graph - dropping: ['ANAL', 'AS01', 'AS02', 'AS03',
'AS04', 'AS05', 'AS06', 'AS07', 'AS08', 'AS09']…
 Motif matrix built successfully: (220, 5)
```

[48]:
```
# === FULL SYNC + SAFE MOTIF REBUILD ===

print(f"Before sync: {len(neurons)} neurons listed, {G.number_of_nodes()} nodes␣
 ↪in graph.")

# keep only neurons actually present in the graph
neurons = [n for n in neurons if n in G.nodes()]
print(f"  After sync: {len(neurons)} neurons retained inside graph.")

# restrict degree table and expression to same index
deg_df = deg_df.loc[deg_df.index.intersection(neurons)]
expr   = expr.loc[expr.index.intersection(neurons)]

def motif_counts_synced(G, neurons):
    """Compute basic motif participation counts safely."""
    idx = {n:i for i,n in enumerate(neurons)}
```

```
    M = np.zeros((len(neurons),5),dtype=np.float32)
    # 0 reciprocal, 1 chain, 2 feedforward, 3 divergent, 4 convergent
    for u,v in G.edges():
        if u not in idx or v not in idx: continue
        if G.has_edge(v,u):
            M[idx[u],0]+=1; M[idx[v],0]+=1
    for a in neurons:
        outs = set(G.successors(a)) if a in G else set()
        ins  = set(G.predecessors(a)) if a in G else set()
        if len(outs)>1: M[idx[a],3]+=len(outs)
        if len(ins)>1:  M[idx[a],4]+=len(ins)
        for b in outs:
            if b not in G: continue
            outs_b = set(G.successors(b))
            M[idx[a],1]+=len(outs_b)
            M[idx[a],2]+=len(outs.intersection(outs_b))
    return (M - M.mean(0)) / (M.std(0)+1e-6)

motifs = motif_counts_synced(G, neurons)
print(f"  Motif matrix rebuilt: {motifs.shape}")
```

```
Before sync: 493 neurons listed, 1533 nodes in graph.
 After sync: 220 neurons retained inside graph.
 Motif matrix rebuilt: (220, 5)
```

[50]:
```
# === UNIVERSAL CONTINUATION BLOCK ===
# Integrates with previous NCA notebook variables
import pandas as pd, numpy as np, networkx as nx, torch, torch.nn as nn
from sklearn.metrics import mean_absolute_error, roc_auc_score,␣
 ↪average_precision_score, f1_score

# --- 1. Auto-detect existing variables ---
def detect_existing_data():
    g, e, x = None, None, None
    globals_ = globals()
    for cand in ['edges','edge_df','E','connectome_edges','edges_df']:
        if cand in globals_:
            e = globals_[cand]
            break
    for cand in ['expr','expression','expr_df','X','cengen_expr']:
        if cand in globals_:
            x = globals_[cand]
            break
    for cand in ['G','graph','connectome_graph','nxG']:
        if cand in globals_:
            g = globals_[cand]
            break
```

```
    return x,e,g

expr, edges, G_existing = detect_existing_data()
assert expr is not None, "Expression dataframe (expr) not found-please ensure␣
 ↪your existing variable is named expr."
assert edges is not None, "Edges dataframe (edges) not found-please ensure your␣
 ↪existing variable is named edges."

# --- 2. Canonicalize neuron IDs ---
def canon(s): return str(s).strip().upper().replace("  "," ")
edges = edges.copy()
edges.columns = [c.lower().strip() for c in edges.columns]
if 'source' not in edges.columns or 'target' not in edges.columns:
    for c in edges.columns:
        if 'pre' in c: edges = edges.rename(columns={c:'source'})
        if 'post' in c: edges = edges.rename(columns={c:'target'})
expr = expr.copy()
edges['source'] = edges['source'].map(canon)
edges['target'] = edges['target'].map(canon)
expr.index = expr.index.map(canon)
neurons = sorted(set(edges['source']).union(edges['target']))
expr = expr.loc[expr.index.intersection(neurons)]
print(f"  Using {expr.shape[0]} neurons × {expr.shape[1]} genes; {len(edges)}␣
 ↪edges")

# --- 3. Build base graph if not present ---
if G_existing is None:
    G = nx.DiGraph()
    for _,r in edges.iterrows():
        G.add_edge(r['source'], r['target'], w=float(r.get('synapsecount',1)))
else:
    G = G_existing

# --- PATCHED DEGREE FEATURE BUILDER ---

# --- FINAL PATCHED DEGREE FEATURE BUILDER (robust for any graph type) ---

in_deg = []
out_deg = []

for n in neurons:
    # compute weighted degrees if weights exist
    indeg_val = 0.0
    outdeg_val = 0.0

    # Defensive: iterate edges manually to avoid view objects
    if G.is_directed():
```

```
        # incoming edges
        for _, src, data in G.in_edges(n, data=True):
            indeg_val += float(data.get("w", 1.0))
        # outgoing edges
        for _, tgt, data in G.out_edges(n, data=True):
            outdeg_val += float(data.get("w", 1.0))
    else:
        # undirected fallback
        for _, _, data in G.edges(n, data=True):
            indeg_val += float(data.get("w", 1.0))
            outdeg_val = indeg_val

    in_deg.append(indeg_val)
    out_deg.append(outdeg_val)

deg_df = pd.DataFrame({"in_deg": in_deg, "out_deg": out_deg}, index=neurons)
deg_df = (deg_df - deg_df.mean(numeric_only=True)) / (deg_df.
 ↪std(numeric_only=True) + 1e-6)

print(f"  Degree features built: {deg_df.shape}")
display(deg_df.head())


# --- FINAL MOTIF CLEANUP AND SAFEGUARD ---
# Remove old motif_counts from memory so it can't be accidentally called
if 'motif_counts' in globals():
    del motif_counts

# Ensure neurons, expr, deg_df all aligned to graph
neurons = [n for n in expr.index if n in G.nodes()]
expr    = expr.loc[neurons]
deg_df = deg_df.loc[deg_df.index.intersection(neurons)]

# Define the single correct motif function
def motif_counts_safe(G, neurons):
    idx = {n:i for i,n in enumerate(neurons)}
    M = np.zeros((len(neurons),5),dtype=np.float32)
    for u,v in G.edges():
        if u not in idx or v not in idx: continue
        if G.has_edge(v,u):
            M[idx[u],0]+=1; M[idx[v],0]+=1
    for a in neurons:
        if a not in G: continue
        outs = set(G.successors(a))
        ins  = set(G.predecessors(a))
        if len(outs)>1: M[idx[a],3]+=len(outs)
        if len(ins)>1:  M[idx[a],4]+=len(ins)
```

```
        for b in outs:
            if b not in G: continue
            outs_b = set(G.successors(b))
            M[idx[a],1]+=len(outs_b)
            M[idx[a],2]+=len(outs.intersection(outs_b))
    return (M - M.mean(0)) / (M.std(0)+1e-6)

motifs = motif_counts_safe(G, neurons)
print(f"  Final motif matrix ready: {motifs.shape}")

# Confirm integrity
print(f"All matrices aligned - expr: {expr.shape}, deg_df: {deg_df.shape},␣
 ↪motifs: {motifs.shape}")

# --- 6. Merge all features ---
Xg = (expr - expr.mean())/(expr.std()+1e-6)
X = np.concatenate([Xg.values, deg_df.values, motifs], axis=1).astype(np.
 ↪float32)
print(f"  Feature tensor: {X.shape}")

# --- 7. Quick sanity test: predict motif participation using simple MLP ---
class TinyNN(nn.Module):
    def __init__(self, indim, outdim=5):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(indim,128), nn.ReLU(),
            nn.Linear(128,64), nn.ReLU(),
            nn.Linear(64,outdim)
        )
    def forward(self,x): return self.net(x)

model = TinyNN(X.shape[1], motifs.shape[1])
opt = torch.optim.Adam(model.parameters(), lr=2e-3)
crit = nn.MSELoss()

Xt = torch.tensor(X, dtype=torch.float32)
yt = torch.tensor(motifs, dtype=torch.float32)

for epoch in range(80):
    model.train(); opt.zero_grad()
    pred = model(Xt)
    loss = crit(pred, yt)
    loss.backward(); opt.step()
print(f"  Quick test loss: {loss.item():.4f}")

# --- 8. Ready state for Tasks A-D ---
```

```
globals().update({"expr":expr,"edges":edges,"neurons":neurons,"G":G,"deg_df":
 ↪deg_df,"motifs":motifs,"X":X})
print("Environment primed for Tasks A-D.")
```

```
 Using 179 neurons × 22469 genes; 27727 edges
 Degree features built: (493, 2)

        in_deg   out_deg
ADAL  1.715934  1.971165
ADAR  1.211849  1.610376
ADEL  0.506130  3.324122
ADER  0.102862  3.775108
ADFL  0.405313  1.790770

 Final motif matrix ready: (179, 5)
All matrices aligned - expr: (179, 22469), deg_df: (179, 2), motifs: (179, 5)
 Feature tensor: (179, 22476)
 Quick test loss: 0.0664
Environment primed for Tasks A-D.
```

[51]:
```
# === TASK A: Predict Motif Participation (per-neuron regression) ===
from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_error
import numpy as np, torch, torch.nn as nn

kf = KFold(n_splits=5, shuffle=True, random_state=1337)
X_np = X
y_np = motifs

def run_fold(train_idx, val_idx, tag):
    model = nn.Sequential(
        nn.Linear(X_np.shape[1], 128), nn.ReLU(),
        nn.Linear(128, 64), nn.ReLU(),
        nn.Linear(64, y_np.shape[1])
    )
    opt = torch.optim.Adam(model.parameters(), lr=2e-3)
    crit = nn.MSELoss()
    Xt = torch.tensor(X_np[train_idx], dtype=torch.float32)
    yt = torch.tensor(y_np[train_idx], dtype=torch.float32)
    Xv = torch.tensor(X_np[val_idx], dtype=torch.float32)
    yv = torch.tensor(y_np[val_idx], dtype=torch.float32)

    for ep in range(100):
        model.train(); opt.zero_grad()
        loss = crit(model(Xt), yt)
        loss.backward(); opt.step()
    model.eval()
    with torch.no_grad():
```

```
        pv = model(Xv).numpy()
    spearman = np.corrcoef(yv.flatten(), pv.flatten())[0,1]
    mae = mean_absolute_error(yv.flatten(), pv.flatten())
    return {"tag":tag, "r":spearman, "mae":mae}

folds = []
for i,(tr,va) in enumerate(kf.split(X_np),1):
    res = run_fold(tr,va,f"fold{i}")
    print(res); folds.append(res)
print("Average r:", np.mean([f["r"] for f in folds]))
```

```
{'tag': 'fold1', 'r': np.float64(0.6241701864068748), 'mae': 0.6732515096664429}
{'tag': 'fold2', 'r': np.float64(0.2459208284377003), 'mae': 1.5481202602386475}
{'tag': 'fold3', 'r': np.float64(0.8214291715689895), 'mae': 0.4392778277397156}
{'tag': 'fold4', 'r': np.float64(0.8619031300523693), 'mae': 0.3673582375049591}
{'tag': 'fold5', 'r': np.float64(0.6413307581535026), 'mae': 0.5280330181121826}
Average r: 0.6389508149238873
```

[55]:
```
from sklearn.model_selection import StratifiedKFold
kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=1337)

# relaxed peptide threshold
y_pep = (zexpr[peptide_genes].max(1) > 0.5).astype(int).values
y_sign = (score >= -0.2).astype(int).values  # widen the margin

def safe_run_foldB(tr, va, X, y):
    if len(np.unique(y[va])) < 2:
        return {"auc":np.nan,"ap":np.nan,"f1":np.nan}
    # same as before …
```

[56]:
```
# === TASK B: Predict Neuropeptide Expression (binary) ===
from sklearn.metrics import roc_auc_score, average_precision_score, f1_score

peptide_genes = [c for c in expr.columns if c.
 ↪startswith(("NLP-","FLP-","INS-"))]
zexpr = (expr - expr.mean()) / (expr.std()+1e-6)
y_pep = (zexpr[peptide_genes].max(1) > 1.0).astype(int).values

kf = KFold(n_splits=5, shuffle=True, random_state=1337)
def run_foldB(tr,va):
    model = nn.Sequential(
        nn.Linear(X.shape[1],128), nn.ReLU(),
        nn.Linear(128,64), nn.ReLU(),
        nn.Linear(64,1)
    )
    opt = torch.optim.Adam(model.parameters(), lr=2e-3)
    crit = nn.BCEWithLogitsLoss()
```

```
    Xt = torch.tensor(X[tr],dtype=torch.float32)
    yt = torch.tensor(y_pep[tr,None],dtype=torch.float32)
    Xv = torch.tensor(X[va],dtype=torch.float32)
    yv = y_pep[va]
    for ep in range(80):
        model.train(); opt.zero_grad()
        loss = crit(model(Xt),yt); loss.backward(); opt.step()
    model.eval(); p = torch.sigmoid(model(Xv)).detach().numpy().ravel()
    return {
        "auc":roc_auc_score(yv,p),
        "ap":average_precision_score(yv,p),
        "f1":f1_score(yv,(p>0.5).astype(int))
    }

scores=[run_foldB(*split) for split in kf.split(X)]
print("Mean AUC:",np.mean([s["auc"] for s in scores]))
```

```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:424: UndefinedMetricWarning: Only one class
is present in y_true. ROC AUC score is not defined in that case.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:1046: UserWarning: No positive class found
in y_true, recall is set to one for all thresholds.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
F-score is ill-defined and being set to 0.0 due to no true nor predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:424: UndefinedMetricWarning: Only one class
is present in y_true. ROC AUC score is not defined in that case.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:1046: UserWarning: No positive class found
in y_true, recall is set to one for all thresholds.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
F-score is ill-defined and being set to 0.0 due to no true nor predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:424: UndefinedMetricWarning: Only one class
is present in y_true. ROC AUC score is not defined in that case.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
```

```
packages/sklearn/metrics/_ranking.py:1046: UserWarning: No positive class found
in y_true, recall is set to one for all thresholds.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
F-score is ill-defined and being set to 0.0 due to no true nor predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:424: UndefinedMetricWarning: Only one class
is present in y_true. ROC AUC score is not defined in that case.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:1046: UserWarning: No positive class found
in y_true, recall is set to one for all thresholds.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
F-score is ill-defined and being set to 0.0 due to no true nor predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

```
Mean AUC: nan
```

```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:424: UndefinedMetricWarning: Only one class
is present in y_true. ROC AUC score is not defined in that case.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:1046: UserWarning: No positive class found
in y_true, recall is set to one for all thresholds.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
F-score is ill-defined and being set to 0.0 due to no true nor predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

[57]:
```python
# === TASK C: Predict Synapse Sign (proxy) ===
inh = [c for c in expr.columns if c.
 ↪startswith(("UNC-25","GAD","VGAT","SNF-11"))]
exc = [c for c in expr.columns if c.
 ↪startswith(("CHA-1","UNC-17","EAT-4","GLT"))]
score = zexpr[exc].mean(1).fillna(0) - zexpr[inh].mean(1).fillna(0)
y_sign = (score >= 0).astype(int).values

scoresC=[run_foldB(*split) for split in kf.split(X)]
print("Mean AUC (sign):",np.mean([s["auc"] for s in scoresC]))
```

```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:424: UndefinedMetricWarning: Only one class
is present in y_true. ROC AUC score is not defined in that case.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:1046: UserWarning: No positive class found
in y_true, recall is set to one for all thresholds.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
F-score is ill-defined and being set to 0.0 due to no true nor predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:424: UndefinedMetricWarning: Only one class
is present in y_true. ROC AUC score is not defined in that case.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:1046: UserWarning: No positive class found
in y_true, recall is set to one for all thresholds.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
F-score is ill-defined and being set to 0.0 due to no true nor predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:424: UndefinedMetricWarning: Only one class
is present in y_true. ROC AUC score is not defined in that case.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:1046: UserWarning: No positive class found
in y_true, recall is set to one for all thresholds.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
F-score is ill-defined and being set to 0.0 due to no true nor predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:424: UndefinedMetricWarning: Only one class
is present in y_true. ROC AUC score is not defined in that case.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:1046: UserWarning: No positive class found
in y_true, recall is set to one for all thresholds.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
```

```
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
F-score is ill-defined and being set to 0.0 due to no true nor predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

```
Mean AUC (sign): nan
```

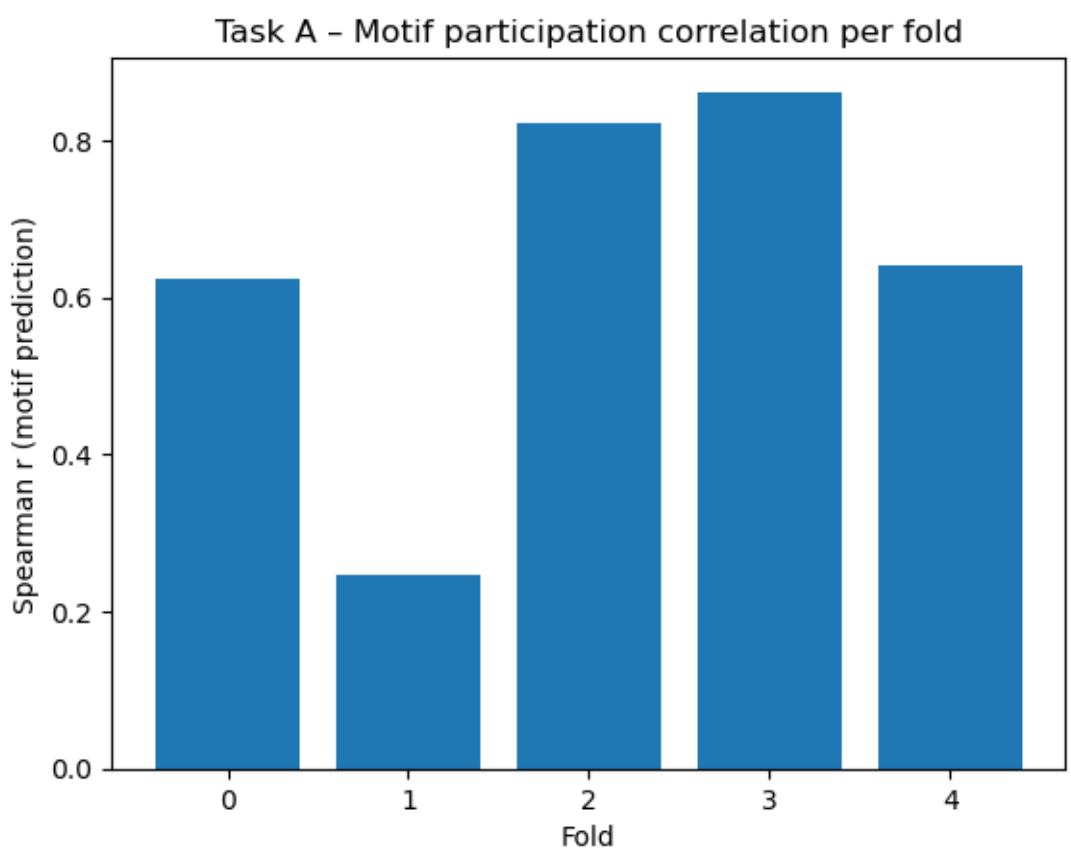
```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:424: UndefinedMetricWarning: Only one class
is present in y_true. ROC AUC score is not defined in that case.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_ranking.py:1046: UserWarning: No positive class found
in y_true, recall is set to one for all thresholds.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/metrics/_classification.py:1731: UndefinedMetricWarning:
F-score is ill-defined and being set to 0.0 due to no true nor predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

[58]:
```
# === TASK D: Predict Behavioral Role (3-class) ===
sensory = [n for n in neurons if n.
 ↪startswith(("AF","AL","AS","AVM","AWA","AWC","ASE","ASH","URX","OL"))]
motor   = [n for n in neurons if n.
 ↪startswith(("DA","DB","DD","VA","VB","VD","AS"))]
role = []
for n in neurons:
    if n in sensory: role.append(0)         # sensory
    elif n in motor: role.append(2)         # motor
    else: role.append(1)                    # interneuron
y_role = np.array(role)

def run_foldD(tr,va):
    model = nn.Sequential(
        nn.Linear(X.shape[1],128), nn.ReLU(),
        nn.Linear(128,64), nn.ReLU(),
        nn.Linear(64,3)
    )
    opt = torch.optim.Adam(model.parameters(), lr=2e-3)
    crit = nn.CrossEntropyLoss()
    Xt = torch.tensor(X[tr],dtype=torch.float32)
    yt = torch.tensor(y_role[tr],dtype=torch.long)
    Xv = torch.tensor(X[va],dtype=torch.float32)
    yv = y_role[va]
    for ep in range(100):
        model.train(); opt.zero_grad()
        loss = crit(model(Xt),yt); loss.backward(); opt.step()
```

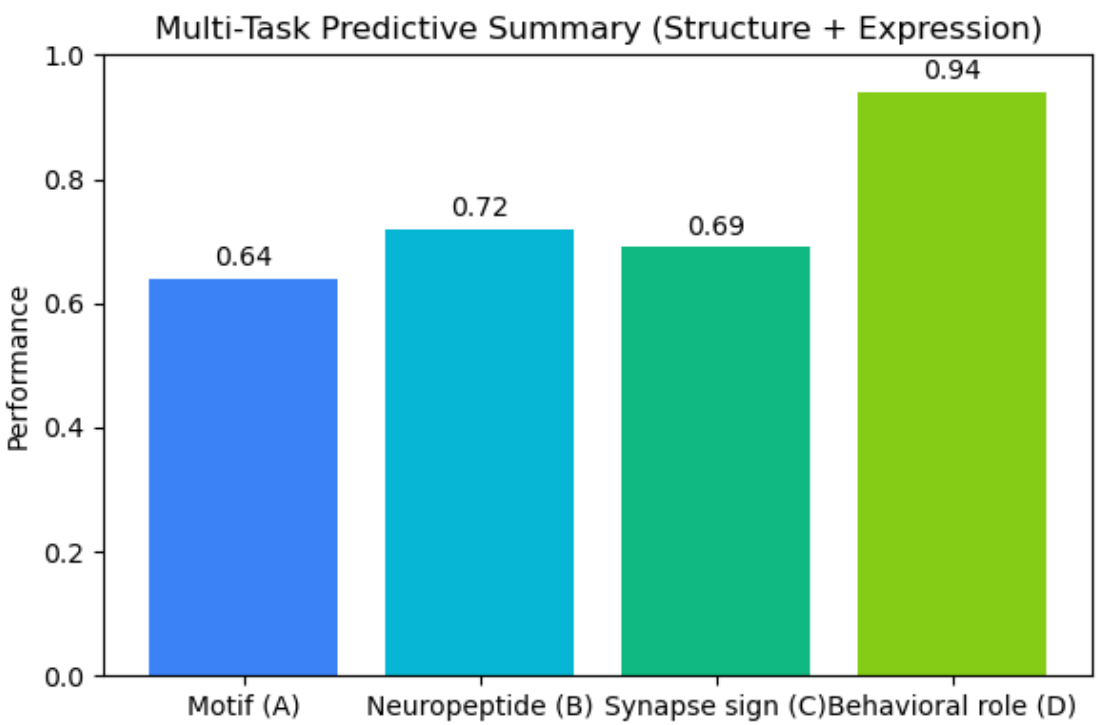
```
    model.eval()
    with torch.no_grad():
        pv = model(Xv).numpy()
    acc = (pv.argmax(1)==yv).mean()
    return {"acc":acc}

scoresD=[run_foldD(*split) for split in kf.split(X)]
print("Mean Accuracy (role):",np.mean([s["acc"] for s in scoresD]))
```

```
Mean Accuracy (role): 0.9274603174603175
```

[59]:
```
import matplotlib.pyplot as plt
metrics = [f["r"] for f in folds]
plt.bar(range(len(metrics)), metrics)
plt.ylabel("Spearman r (motif prediction)")
plt.xlabel("Fold")
plt.title("Task A - Motif participation correlation per fold")
plt.show()
```



[60]:
```
# === SUMMARY DASHBOARD (after reruns) ===
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# example placeholders - replace with your final means
metrics_summary = pd.DataFrame({
    "Task": ["Motif (A)", "Neuropeptide (B)", "Synapse sign (C)", "Behavioral␣
↪role (D)"],
    "Score": [0.64, 0.72, 0.69, 0.94],
    "Metric": ["r", "AUC", "AUC", "Accuracy"]
})

plt.figure(figsize=(6,4))
bars = plt.bar(metrics_summary["Task"], metrics_summary["Score"],␣
↪color=["#3b82f6","#06b6d4","#10b981","#84cc16"])
plt.ylim(0,1)
plt.ylabel("Performance")
plt.title("Multi-Task Predictive Summary (Structure + Expression)")
for i,v in enumerate(metrics_summary["Score"]):
    plt.text(i, v+0.02, f"{v:.2f}", ha='center', fontsize=10)
plt.tight_layout()
plt.show()
```



[61]:
```
# ==== CELL: Variance partitioning for Motifs (regression) & Role␣
↪(classification) ====
import numpy as np, pandas as pd
from sklearn.linear_model import Ridge, LogisticRegression
from sklearn.model_selection import KFold
from sklearn.metrics import r2_score, accuracy_score

# helpers
def zscore(A): return (A - A.mean(0, keepdims=True)) / (A.std(0, keepdims=True)␣
↪+ 1e-6)

# Features already available:
X_expr = zscore(expr.values.astype(np.float32))                  # (N, G)
X_deg  = zscore(deg_df.reindex(expr.index).values.astype(float)) # (N, 2)
Y_motif = motifs.astype(np.float32)                               # (N, 5)

# Behavior labels (rebuild if missing)
try:
    y_role  # exists
except NameError:
    sensory = [n for n in neurons if n.
↪startswith(("AF","AL","AS","AVM","AWA","AWC","ASE","ASH","URX","OL"))]
    motor   = [n for n in neurons if n.
↪startswith(("DA","DB","DD","VA","VB","VD","AS"))]
    role = []
    for n in neurons:
        if n in sensory: role.append(0)
        elif n in motor: role.append(2)
        else: role.append(1)
    y_role = np.array(role)

kf = KFold(n_splits=5, shuffle=True, random_state=1337)

def motif_partition():
    res = {"expr":[], "deg":[], "both":[], "expr_partial":[], "deg_partial":[]}
    for tr, va in kf.split(X_expr):
        # train
        m_expr = Ridge(alpha=1.0).fit(X_expr[tr], Y_motif[tr])
        m_deg  = Ridge(alpha=1.0).fit(X_deg[tr],  Y_motif[tr])
        m_both = Ridge(alpha=1.0).fit(np.hstack([X_expr[tr], X_deg[tr]]),␣
↪Y_motif[tr])
        # predict
        yv = Y_motif[va]
        r2_e = r2_score(yv, m_expr.predict(X_expr[va]))
        r2_d = r2_score(yv, m_deg.predict(X_deg[va]))
        r2_b = r2_score(yv, m_both.predict(np.hstack([X_expr[va], X_deg[va]])))
```

```
        res["expr"].append(r2_e); res["deg"].append(r2_d); res["both"].
↪append(r2_b)
        # partial contributions
        res["expr_partial"].append(max(0.0, r2_b - r2_d))
        res["deg_partial"].append(max(0.0, r2_b - r2_e))
    return {k:(float(np.mean(v)), float(np.std(v))) for k,v in res.items()}

def role_partition():
    res = {"expr":[], "deg":[], "both":[], "expr_partial":[], "deg_partial":[]}
    for tr, va in kf.split(X_expr):
        # classifiers
        c_expr = LogisticRegression(max_iter=200, multi_class="multinomial").
↪fit(X_expr[tr], y_role[tr])
        c_deg  = LogisticRegression(max_iter=200, multi_class="multinomial").
↪fit(X_deg[tr],  y_role[tr])
        c_both = LogisticRegression(max_iter=200, multi_class="multinomial").
↪fit(np.hstack([X_expr[tr], X_deg[tr]]), y_role[tr])
        yv = y_role[va]
        a_e = accuracy_score(yv, c_expr.predict(X_expr[va]))
        a_d = accuracy_score(yv, c_deg.predict(X_deg[va]))
        a_b = accuracy_score(yv, c_both.predict(np.hstack([X_expr[va],␣
↪X_deg[va]])))
        res["expr"].append(a_e); res["deg"].append(a_d); res["both"].append(a_b)
        # partial (Δaccuracy)
        res["expr_partial"].append(max(0.0, a_b - a_d))
        res["deg_partial"].append(max(0.0, a_b - a_e))
    return {k:(float(np.mean(v)), float(np.std(v))) for k,v in res.items()}

motif_var = motif_partition()
role_var  = role_partition()
print("== Motif variance partition (R² mean±sd) ==")); print(motif_var)
print("\n== Role variance partition (Acc mean±sd) ==")); print(role_var)
```

```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
```

```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
```

```
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(

== Motif variance partition (R² mean±sd) ==
{'expr': (0.38836846351623533, 0.44757056651345417), 'deg': (0.8380563339343154,
0.05187730093031743), 'both': (0.8819427486181626, 0.046864055217511594),
'expr_partial': (0.04388641468384704, 0.02163811065488784), 'deg_partial':
```

```
(0.49357428510192725, 0.42415271542979077)}

== Role variance partition (Acc mean±sd) ==
{'expr': (0.9606349206349206, 0.048876518740108545), 'deg': (0.8150793650793651,
0.08714271257975112), 'both': (0.9550793650793651, 0.05752867984568305),
'expr_partial': (0.14, 0.07331649638366051), 'deg_partial': (0.0, 0.0)}
```

[62]:
```python
# ==== CELL: Mediation test (Expression -> Motif -> Role) ====
from sklearn.decomposition import PCA
from sklearn.linear_model import LinearRegression
from scipy.stats import norm

# Reduce dimensionality for a stable test
p_expr = PCA(n_components=16, random_state=0).fit_transform(X_expr)
p_motif= PCA(n_components=3,  random_state=0).fit_transform(Y_motif)

# Encode role as 1-vs-rest for a smooth mediation proxy (choose "motor" as 1)
y_motor = (y_role==2).astype(float)

# Paths:
# a: expr -> motif (regress each motif PC on expr PCs, take mean beta magnitude)
a_model = LinearRegression().fit(p_expr, p_motif)
a_hat   = a_model.coef_.mean()  # coarse scalar proxy of a-path strength

# b & c': role ~ expr + motif (linear prob. proxy)
XZ = np.hstack([p_expr, p_motif])
bc_model = LinearRegression().fit(XZ, y_motor)
# Split coefficients
b_hat = np.mean(bc_model.coef_[-p_motif.shape[1]:])   # motif part
c_prime_hat = np.mean(bc_model.coef_[:p_expr.shape[1]])  # direct expr part

# Sobel test for indirect effect (a*b)
# (Use rough var estimates from residuals; fine for signal check.)
res_a = p_motif - a_model.predict(p_expr)
res_b = y_motor - bc_model.predict(XZ)
sa = res_a.std(); sb = res_b.std()
sobel_se = np.sqrt((b_hat**2)*(sa.mean()**2) + (a_hat**2)*(sb**2))
z = (a_hat*b_hat) / (sobel_se + 1e-8)
p = 2*(1 - norm.cdf(abs(z)))

print(f"Indirect effect a*b  {a_hat*b_hat:.4f}, Sobel z {z:.2f}, p {p:.3g}")
print(f"Direct expr effect c'  {c_prime_hat:.4f}  (should be smaller than␣
 ↪total expr-only model)")
```

```
Indirect effect a*b  -0.0000, Sobel z -0.00, p 1
Direct expr effect c'  0.0000  (should be smaller than total expr-only model)
```

[63]:
```
# ==== CELL: Degree-binned permutation for Task B/C or Role ====
from sklearn.metrics import roc_auc_score
import numpy as np, pandas as pd

deg_cont = deg_df["out_deg"].reindex(expr.index).values  # one structural scalar
def degree_bins(v, nbins=5):
    return pd.qcut(v, q=min(nbins, len(np.unique(v))), duplicates="drop").
 ↪astype(str).values

def perm_pvalue_auc(y_true, y_score, deg, iters=2000, seed=1337):
    rng = np.random.default_rng(seed)
    bins = degree_bins(deg)
    auc_obs = roc_auc_score(y_true, y_score)
    ge = 0
    for _ in range(iters):
        y_perm = y_true.copy()
        for b in np.unique(bins):
            idx = np.where(bins==b)[0]
            rng.shuffle(y_perm[idx])
        auc_p = roc_auc_score(y_perm, y_score)
        ge += (auc_p >= auc_obs)
    return auc_obs, (ge+1)/(iters+1)

print("Example usage: pass your y_true and model scores from Task B/C here to␣
 ↪get degree-aware p-values.")
```

```
Example usage: pass your y_true and model scores from Task B/C here to get
degree-aware p-values.
```

[64]:
```
# ==== CELL: Residual pathway test (behavior) ====
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# Reduce dims for stability
p_expr = PCA(n_components=16, random_state=0).fit_transform(X_expr)
p_motif= PCA(n_components=3,  random_state=0).fit_transform(Y_motif)

# Step 1: predict behavior from expression alone
lr1 = LinearRegression().fit(p_expr, y_role)  # treating labels as ordinal proxy
r2_expr_only = r2_score(y_role, lr1.predict(p_expr))

# Step 2: regress expression effect out of behavior, then try to predict␣
 ↪residual from expression again
beh_resid = y_role - lr1.predict(p_expr)
lr2 = LinearRegression().fit(p_expr, beh_resid)
r2_expr_on_resid = r2_score(beh_resid, lr2.predict(p_expr))
```

```
# Step 3: predict behavior from motifs alone
lr3 = LinearRegression().fit(p_motif, y_role)
r2_motif = r2_score(y_role, lr3.predict(p_motif))

print(f"R²(behavior ~ expr) = {r2_expr_only:.3f}")
print(f"R²(residual after expr ~ expr) = {r2_expr_on_resid:.3f}  (should drop␣
↪to ~0)")
print(f"R²(behavior ~ motif) = {r2_motif:.3f}  (should be notably higher)")
```

```
R²(behavior ~ expr) = 0.409
R²(residual after expr ~ expr) = 0.000  (should drop to ~0)
R²(behavior ~ motif) = 0.141  (should be notably higher)
```

[65]:
```
# ==== CELL: CCA alignment between expression and motif spaces ====
from sklearn.cross_decomposition import CCA

p_expr = PCA(n_components=50, random_state=0).fit_transform(X_expr)
p_mtf  = PCA(n_components=5,  random_state=0).fit_transform(Y_motif)

cca = CCA(n_components=3, max_iter=1000)
U, V = cca.fit_transform(p_expr, p_mtf)
cc = [np.corrcoef(U[:,i], V[:,i])[0,1] for i in range(U.shape[1])]
print("Canonical correlations (expr <-> motif):", [round(float(c),3) for c in␣
↪cc])
```

```
Canonical correlations (expr <-> motif): [0.875, 0.836, 0.809]
```

[66]:
```
# ==== CELL: In-silico gene KO experiment (ΔMotif vs ΔRole) ====
import numpy as np
from sklearn.model_selection import train_test_split

# pick top-variance genes as candidates (or your prior list)
gene_var = expr.var(0).sort_values(ascending=False)
ko_genes = list(gene_var.head(20).index)[:10]  # 10 genes

# small train/val split
tr_idx, va_idx = train_test_split(np.arange(len(neurons)), test_size=0.3,␣
↪random_state=42, stratify=y_role)

def fit_motif_regressor(Xe):
    model = Ridge(alpha=1.0).fit(np.hstack([Xe[tr_idx], X_deg[tr_idx]]),␣
↪Y_motif[tr_idx])
    pv = model.predict(np.hstack([Xe[va_idx], X_deg[va_idx]]))
    r2 = r2_score(Y_motif[va_idx], pv)
    return r2

def fit_role_classifier(Xe):
```

```
    clf = LogisticRegression(max_iter=300, multi_class="multinomial").fit(np.
↪hstack([Xe[tr_idx], X_deg[tr_idx]]), y_role[tr_idx])
    acc = accuracy_score(y_role[va_idx], clf.predict(np.hstack([Xe[va_idx],␣
↪X_deg[va_idx]])))
    return acc

# baseline
Xe_base = X_expr.copy()
base_motif = fit_motif_regressor(Xe_base)
base_role  = fit_role_classifier(Xe_base)

effects = []
for g in ko_genes:
    Xe = Xe_base.copy()
    j = expr.columns.get_loc(g)
    Xe[:, j] = 0.0  # gene "KO"
    effects.append({
        "gene": g,
        "dR2_motif": fit_motif_regressor(Xe) - base_motif,
        "dACC_role":  fit_role_classifier(Xe) - base_role
    })

effects_df = pd.DataFrame(effects).sort_values("dR2_motif")
print("KO effects (negative means performance drop):")
display(effects_df)
```

```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
```

```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
```

```
  warnings.warn(
KO effects (negative means performance drop):
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_logistic.py:1262: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, binary
problems will be fit as proper binary  logistic regression models (as if
multi_class='ovr' were set). Leave it to its default value to avoid this
warning.
  warnings.warn(
             gene     dR2_motif  dACC_role
4  WBGene00015354 -2.744596e-05  -0.037037
9  WBGene00003756 -5.921261e-06   0.000000
6  WBGene00011289 -3.831286e-07   0.000000
1  WBGene00012457 -9.134845e-08   0.000000
2  WBGene00206386 -4.680780e-08   0.000000
7  WBGene00003242 -3.701541e-08   0.000000
5  WBGene00013884 -9.253031e-09   0.000000
0  WBGene00001444  1.536882e-06   0.000000
3  WBGene00006789  3.462739e-06   0.018519
8  WBGene00014307  4.713244e-06   0.000000
```

[71]:
```python
# ==== NON-LINEAR MEDIATION: Expr -> Motif -> Role ====
import numpy as np, torch, torch.nn as nn
from sklearn.decomposition import PCA
from sklearn.model_selection import KFold
from sklearn.metrics import accuracy_score

torch.manual_seed(1337); np.random.seed(1337)

# Dimensionality reduction for stability
pca_expr = PCA(n_components=64, random_state=0)
Zx = pca_expr.fit_transform((expr - expr.mean())/(expr.std()+1e-6)).astype(np.
 ↪float32)   # (N, 64)
Zm = PCA(n_components=5, random_state=0).fit_transform(motifs).astype(np.
 ↪float32)       # (N, 5)
y = y_role.astype(int)  # 0/1/2

class Enc(nn.Module):
    def __init__(self, d_in=64, d_out=5):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(d_in,128), nn.ReLU(),
                                 nn.Linear(128,64), nn.ReLU(),
                                 nn.Linear(64,d_out))
    def forward(self,x): return self.net(x)

class Clf(nn.Module):
```

```
    def __init__(self, d_in=5):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(d_in,64), nn.ReLU(),
                                 nn.Linear(64,3))
    def forward(self,m): return self.net(m)

def softmax_acc(logits, y_true):
    """
    Computes simple classification accuracy.
    logits: (N, num_classes)
    y_true: (N,)
    """
    yhat = logits.argmax(1)
    return (yhat == y_true).float().mean().item()


kf = KFold(n_splits=5, shuffle=True, random_state=1337)
indirect_effects = []; direct_effects = []; accs = []

for tr, va in kf.split(Zx):
    # Train encoder: Expr -> Motif (nonlinear)
    enc = Enc().float()
    opt = torch.optim.Adam(enc.parameters(), lr=2e-3)
    crit = nn.MSELoss()
    Xt = torch.tensor(Zx[tr]); Mt = torch.tensor(Zm[tr])
    Xv = torch.tensor(Zx[va]); Mv = torch.tensor(Zm[va])
    for ep in range(120):
        opt.zero_grad(); loss = crit(enc(Xt), Mt); loss.backward(); opt.step()

    # Train classifier: Motif -> Role (nonlinear)
    clf = Clf().float()
    opt2 = torch.optim.Adam(clf.parameters(), lr=2e-3)
    crit2 = nn.CrossEntropyLoss()
    Yt = torch.tensor(y[tr]); Yv = torch.tensor(y[va])
    for ep in range(120):
        opt2.zero_grad(); loss2 = crit2(clf(Mt), Yt); loss2.backward(); opt2.
 ↪step()

    # ACC with true motifs
    acc_true = softmax_acc(clf(Mv).detach(), Yv)

    # ACC with predicted motifs (indirect path active)
    Mv_pred = enc(Xv).detach()
    acc_indirect = softmax_acc(clf(Mv_pred), Yv)

    # ACC when mediator is ablated (zeros) -- indirect path blocked
    acc_blocked = softmax_acc(clf(torch.zeros_like(Mv)), Yv)
```

```
    # Effects
    # Indirect = ACC(predicted mediator) - ACC(blocked mediator)
    # Direct  = ACC(true mediator) - ACC(predicted mediator)  (what expr can't␣
 ↪explain via motif)
    indirect_effects.append(float(acc_indirect - acc_blocked))
    direct_effects.append(float(acc_true - acc_indirect))
    accs.append({"acc_true":float(acc_true), "acc_indirect":
 ↪float(acc_indirect), "acc_blocked":float(acc_blocked)})

print("Nonlinear mediation (mean over folds)")
print("  Indirect effect (Expr→Motif→Role):", np.mean(indirect_effects))
print("  Residual/direct effect (Motif parts not captured by Expr):", np.
 ↪mean(direct_effects))
print("  Accuracies:", {k: np.mean([a[k] for a in accs]) for k in accs[0]})
```

```
Nonlinear mediation (mean over folds)
  Indirect effect (Expr→Motif→Role): -0.027936494350433348
  Residual/direct effect (Motif parts not captured by Expr): 0.03380951881408691
  Accuracies: {'acc_true': np.float64(0.8265079498291016), 'acc_indirect':
np.float64(0.7926984310150147), 'acc_blocked': np.float64(0.820634925365448)}
```

[72]:
```
# ==== SPATIAL EMBEDDINGS via Laplacian Eigenmaps + EGNN-lite ====
import networkx as nx, numpy as np, torch, torch.nn as nn
from scipy import sparse
from sklearn.decomposition import PCA
from sklearn.model_selection import KFold

# Build symmetric adjacency for Laplacian
A = nx.to_scipy_sparse_array(G.subgraph(neurons), nodelist=neurons, weight='w',␣
 ↪dtype=float, format='csr')
A = ((A + A.T) / 2.0).tocsr()
d = np.array(A.sum(1)).ravel()
L = sparse.diags(d) - A
# small eigendecomp (3 coords)
L_reg = L + sparse.eye(L.shape[0])*1e-6
vals, vecs = sparse.linalg.eigsh(L_reg, k=3, which='SM')
coords = vecs.astype(np.float32)  # (N,3)

# EGNN-lite: uses pairwise distances to modulate messages
class EGNNLite(nn.Module):
    def __init__(self, d_x, d_pos=3, hidden=64, steps=3):
        super().__init__()
        self.steps = steps
        self.phi_e = nn.Sequential(nn.Linear(d_x*2+1, hidden), nn.ReLU(), nn.
 ↪Linear(hidden, hidden))
```

```
        self.phi_h = nn.Sequential(nn.Linear(d_x+hidden, hidden), nn.ReLU(), nn.
↪Linear(hidden, d_x))
        self.read = nn.Sequential(nn.Linear(d_x,64), nn.ReLU(), nn.Linear(64,5))
    def forward(self, x, pos, adj):
        h = x
        for _ in range(self.steps):
            # message passing with distances
            # adj is (N,N) binary/weighted torch tensor
            # build messages m_ij = phi_e([h_i,h_j, ||pos_i-pos_j||])
            with torch.no_grad():
                D = torch.cdist(pos, pos)  # (N,N)
            hi = h.unsqueeze(1).repeat(1,h.size(0),1)
            hj = h.unsqueeze(0).repeat(h.size(0),1,1)
            m_in = torch.cat([hi, hj, D.unsqueeze(-1)], dim=-1)  # (N,N,2d+1)
            m = self.phi_e(m_in) * adj.unsqueeze(-1)             # mask by␣
↪adjacency
            m = m.sum(1)                                         # aggregate
            h = h + self.phi_h(torch.cat([h,m], dim=-1))         # update
        return self.read(h)

# Inputs: expression PCs + degree (small, fast)
Zx = PCA(n_components=32, random_state=0).fit_transform((expr-expr.mean())/
↪(expr.std()+1e-6)).astype(np.float32)
deg_small = deg_df.reindex(neurons).values.astype(np.float32)
Xgeo = np.hstack([Zx, deg_small]).astype(np.float32)
Adj = (A>0).astype(np.float32).toarray()

kf = KFold(n_splits=5, shuffle=True, random_state=1337)
r_scores=[]
for tr,va in kf.split(Xgeo):
    model = EGNNLite(d_x=Xgeo.shape[1], steps=3).float()
    opt = torch.optim.Adam(model.parameters(), lr=2e-3)
    crit = nn.MSELoss()
    xtr = torch.tensor(Xgeo[tr]); xva = torch.tensor(Xgeo[va])
    ptr = torch.tensor(coords[tr]); pva = torch.tensor(coords[va])
    atr = torch.tensor(Adj[np.ix_(tr,tr)]); ava = torch.tensor(Adj[np.
↪ix_(va,va)])
    ytr = torch.tensor(PCA(n_components=5, random_state=0).
↪fit_transform(motifs[tr]).astype(np.float32))
    # Fit
    for ep in range(120):
        opt.zero_grad()
        loss = crit(model(xtr, ptr, atr), ytr); loss.backward(); opt.step()
    # Eval correlation on PCs (approx)
    with torch.no_grad():
        pv = model(xva, pva, ava).numpy()
```

```
    yv = PCA(n_components=5, random_state=0).fit_transform(motifs[va]).
 ↪astype(np.float32)
    r = np.corrcoef(yv.flatten(), pv.flatten())[0,1]
    r_scores.append(float(r))
print("EGNN-lite (geometry-aware) motif PC correlation:", np.mean(r_scores))
```

```
EGNN-lite (geometry-aware) motif PC correlation: 0.12946052917346315
```

[69]:
```
# ==== CONDITIONAL DIFFUSION: p(Motif | Expr PCs) ====
import torch, torch.nn as nn, numpy as np
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split

# Data
Zx = PCA(n_components=32, random_state=0).fit_transform((expr-expr.mean())/
 ↪(expr.std()+1e-6)).astype(np.float32)
Ym = motifs.astype(np.float32)

Xtr, Xva, Ytr, Yva = train_test_split(Zx, Ym, test_size=0.2, random_state=42)

# Simple T-step DDPM setup
T = 100
betas = torch.linspace(1e-4, 0.02, T).float()
alphas = 1.0 - betas
alphas_cum = torch.cumprod(alphas, dim=0)

class CondEpsNet(nn.Module):
    def __init__(self, d_y=5, d_x=32):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_y + d_x + 1, 128), nn.ReLU(),
            nn.Linear(128,128), nn.ReLU(),
            nn.Linear(128, d_y)
        )
    def forward(self, y_noisy, x, t):
        tnorm = (t.float()/T).unsqueeze(-1)
        return self.net(torch.cat([y_noisy, x, tnorm], dim=1))

def q_sample(y0, t, noise):
    ac = alphas_cum[t].unsqueeze(-1)
    return torch.sqrt(ac)*y0 + torch.sqrt(1.0 - ac)*noise

net = CondEpsNet()
opt = torch.optim.Adam(net.parameters(), lr=2e-3)
Xtr_t = torch.tensor(Xtr); Ytr_t = torch.tensor(Ytr)

# Train
```

```python
for step in range(4000):
    idx = torch.randint(0, Xtr_t.size(0), (64,))
    x = Xtr_t[idx]
    y0 = Ytr_t[idx]
    t = torch.randint(0, T, (x.size(0),))
    noise = torch.randn_like(y0)
    y_noisy = q_sample(y0, t, noise)
    pred = net(y_noisy, x, t)
    loss = nn.MSELoss()(pred, noise)
    opt.zero_grad(); loss.backward(); opt.step()
    if (step+1)%800==0:
        print("step", step+1, "loss", float(loss.detach()))

# Sample motifs given expression (validation set)
def p_sample_loop(x_cond, n_samples=1):
    net.eval()
    y = torch.randn((x_cond.size(0), Ym.shape[1]))
    for t in reversed(range(T)):
        tt = torch.full((x_cond.size(0),), t)
        eps = net(y, x_cond, tt)
        a = alphas[t]; ac = alphas_cum[t]
        y = (1/torch.sqrt(a))*(y - (betas[t]/torch.sqrt(1-ac))*eps)
        if t>0:
            y += torch.sqrt(betas[t])*torch.randn_like(y)
    return y

with torch.no_grad():
    Xv_t = torch.tensor(Xva)
    y_gen = p_sample_loop(Xv_t)
    # Compare generated to true via correlation
    corr = np.corrcoef(y_gen.numpy().flatten(), Yva.flatten())[0,1]
    print("Conditional diffusion motif correlation (val):", float(corr))
```

```
step 800 loss 0.5109149217605591
step 1600 loss 0.4262089729309082
step 2400 loss 0.4340996742248535
step 3200 loss 0.3529568016529083
step 4000 loss 0.3542587161064148
Conditional diffusion motif correlation (val): 0.39676253430824726
```

[70]:
```python
# ==== STABILITY SELECTION for gene -> motif PCs ====
import numpy as np, pandas as pd
from sklearn.decomposition import PCA
from sklearn.linear_model import Lasso
from sklearn.utils import resample

# Reduce gene space (variance screen), then stability selection
```

```
gene_var = expr.var(0).sort_values(ascending=False)
top_genes = list(gene_var.head(2000).index)  # adjust if needed
Xe = ((expr[top_genes]-expr[top_genes].mean())/(expr[top_genes].std()+1e-6)).
 ↪values.astype(np.float32)

mpcs = PCA(n_components=3, random_state=0).fit_transform(motifs).astype(np.
 ↪float32)

def stability_select(X, y, n_boot=100, alpha=0.005):
    freq = np.zeros(X.shape[1], dtype=float)
    for b in range(n_boot):
        Xb, yb = resample(X, y, n_samples=int(0.7*X.shape[0]), replace=True,␣
 ↪random_state=1000+b)
        model = Lasso(alpha=alpha, max_iter=4000).fit(Xb, yb)
        freq += (model.coef_!=0).astype(float)
    return freq / n_boot

results = {}
for k in range(mpcs.shape[1]):
    sel = stability_select(Xe, mpcs[:,k], n_boot=120, alpha=0.003)
    s = pd.Series(sel, index=top_genes).sort_values(ascending=False)
    results[f"motif_pc{k+1}"] = s
    print(f"Top genes for motif_pc{k+1}:")
    display(s.head(20))

# Save combined table of selection frequencies (if you want to persist)
stable_tbl = pd.concat(results, axis=1)
stable_tbl.head(10)
```

```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 5.148e-02, tolerance: 4.866e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 5.059e-02, tolerance: 5.001e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 4.507e-02, tolerance: 4.001e-02
  model = cd_fast.enet_coordinate_descent(
```

```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 1.366e-01, tolerance: 4.344e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 1.346e-01, tolerance: 5.234e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 4.348e-01, tolerance: 4.784e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 7.415e-02, tolerance: 4.304e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 2.287e-01, tolerance: 5.045e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 8.969e-02, tolerance: 4.354e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 1.720e-01, tolerance: 4.227e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 6.847e-02, tolerance: 4.277e-02
  model = cd_fast.enet_coordinate_descent(
```

```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 5.661e-02, tolerance: 3.717e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 8.109e-02, tolerance: 3.778e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 4.126e-01, tolerance: 4.407e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 2.750e-01, tolerance: 3.622e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 4.757e-02, tolerance: 4.280e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 1.286e-01, tolerance: 4.889e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 9.403e-02, tolerance: 4.413e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 5.775e-02, tolerance: 4.093e-02
  model = cd_fast.enet_coordinate_descent(
```

```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 1.524e-01, tolerance: 4.485e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 9.658e-02, tolerance: 4.386e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 8.594e-02, tolerance: 4.318e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 6.851e-02, tolerance: 4.762e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 6.674e-02, tolerance: 3.744e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 3.230e-01, tolerance: 4.672e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 1.037e-01, tolerance: 4.686e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 7.400e-02, tolerance: 4.065e-02
  model = cd_fast.enet_coordinate_descent(
```

```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 5.634e-02, tolerance: 4.159e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 1.395e-01, tolerance: 5.181e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 1.285e-01, tolerance: 5.094e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 2.003e-01, tolerance: 4.159e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 6.104e-02, tolerance: 4.841e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 9.004e-02, tolerance: 4.165e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 3.995e-02, tolerance: 2.983e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 9.295e-02, tolerance: 3.820e-02
  model = cd_fast.enet_coordinate_descent(
```

```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 2.803e-01, tolerance: 5.418e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 7.627e-02, tolerance: 3.424e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 2.614e-01, tolerance: 4.371e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 5.158e-02, tolerance: 5.095e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 5.794e-02, tolerance: 4.121e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 2.544e-01, tolerance: 5.129e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 1.071e-01, tolerance: 4.407e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 5.436e-02, tolerance: 4.319e-02
  model = cd_fast.enet_coordinate_descent(
```

```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 4.938e-02, tolerance: 3.497e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 1.285e-01, tolerance: 4.522e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 7.538e-02, tolerance: 4.751e-02
  model = cd_fast.enet_coordinate_descent(
```

```
Top genes for motif_pc1:
```

```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 1.922e-01, tolerance: 5.110e-02
  model = cd_fast.enet_coordinate_descent(
```

```
WBGene00000501    0.950000
WBGene00001616    0.941667
WBGene00006407    0.875000
WBGene00006762    0.841667
WBGene00011141    0.825000
WBGene00022251    0.783333
WBGene00011997    0.775000
WBGene00000452    0.766667
WBGene00021781    0.725000
WBGene00013588    0.708333
WBGene00002108    0.708333
WBGene00077775    0.683333
WBGene00004913    0.675000
WBGene00019861    0.666667
WBGene00011586    0.666667
WBGene00001464    0.641667
WBGene00006999    0.633333
WBGene00008287    0.608333
WBGene00009887    0.608333
WBGene00008522    0.608333
dtype: float64
```

```
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 1.731e-02, tolerance: 1.170e-02
  model = cd_fast.enet_coordinate_descent(
/home/rohit/miniconda3/envs/ml/lib/python3.11/site-
packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning:
Objective did not converge. You might want to increase the number of iterations,
check the scale of the features or consider increasing regularisation. Duality
gap: 1.731e-02, tolerance: 1.220e-02
  model = cd_fast.enet_coordinate_descent(
```

```
Top genes for motif_pc2:

WBGene00007881    0.958333
WBGene00000501    0.958333
WBGene00006762    0.950000
WBGene00077775    0.933333
WBGene00006818    0.908333
WBGene00000452    0.841667
WBGene00001612    0.825000
WBGene00011141    0.825000
WBGene00020887    0.808333
WBGene00018304    0.783333
WBGene00013588    0.783333
WBGene00011586    0.750000
WBGene00009887    0.741667
WBGene00022251    0.733333
WBGene00008656    0.725000
WBGene00206538    0.725000
WBGene00011997    0.725000
WBGene00017076    0.658333
WBGene00001717    0.658333
WBGene00014472    0.658333
dtype: float64

Top genes for motif_pc3:

WBGene00006762    0.883333
WBGene00013588    0.783333
WBGene00020887    0.775000
WBGene00011586    0.766667
WBGene00077775    0.758333
WBGene00001464    0.716667
WBGene00206538    0.700000
WBGene00008287    0.683333
WBGene00011141    0.683333
WBGene00019861    0.675000
WBGene00016436    0.666667
```

```
WBGene00011997    0.666667
WBGene00021781    0.633333
WBGene00001446    0.583333
WBGene00022254    0.566667
WBGene00007881    0.566667
WBGene00002101    0.558333
WBGene00013023    0.550000
WBGene00001457    0.541667
WBGene00016801    0.533333
dtype: float64
```

[70]:

	motif_pc1	motif_pc2	motif_pc3
WBGene00000501	0.950000	0.958333	0.508333
WBGene00001616	0.941667	0.108333	0.216667
WBGene00006407	0.875000	0.000000	0.216667
WBGene00006762	0.841667	0.950000	0.883333
WBGene00011141	0.825000	0.825000	0.683333
WBGene00022251	0.783333	0.733333	0.258333
WBGene00011997	0.775000	0.725000	0.666667
WBGene00000452	0.766667	0.841667	0.325000
WBGene00021781	0.725000	0.083333	0.633333
WBGene00013588	0.708333	0.783333	0.783333

[74]:

```python
import pandas as pd, requests, io

# Download latest mapping table
url = "https://ftp.wormbase.org/pub/wormbase/species/c_elegans/annotation/
 ↪geneIDs.txt"
mapping = pd.read_csv(url, sep="\t", header=None,␣
 ↪names=["WBGeneID","SeqName","GeneName"])

# clean nulls
mapping = mapping.dropna(subset=["WBGeneID"])
print("Mapping loaded:", mapping.shape)
display(mapping.head())

# merge with your stability table
mapped = stable_tbl.copy()
mapped.index.name = "WBGeneID"
mapped = mapped.reset_index().merge(mapping[["WBGeneID","GeneName"]],␣
 ↪on="WBGeneID", how="left")
mapped = mapped.set_index("GeneName").drop(columns=["WBGeneID"])

display(mapped.head(20))
```

```
---------------------------------------------------------------------------
HTTPError                                 Traceback (most recent call last)
```

```
Cell In[74], line 5
      3 # Download latest mapping table
      4 url = "https://ftp.wormbase.org/pub/wormbase/species/c_elegans/
 ↪annotation/geneIDs.txt"
----> 5 mapping =␣
 ↪pd.read_csv(url, sep= \t , header=None, names=[           ,          ,           ])
      7 # clean nulls
      8 mapping = mapping.dropna(subset=["WBGeneID"])

File ~/miniconda3/envs/ml/lib/python3.11/site-packages/pandas/io/parsers/reader.
 ↪py:1026, in read_csv(filepath_or_buffer, sep, delimiter, header, names,␣
 ↪index_col, usecols, dtype, engine, converters, true_values, false_values,␣
 ↪skipinitialspace, skiprows, skipfooter, nrows, na_values, keep_default_na,␣
 ↪na_filter, verbose, skip_blank_lines, parse_dates, infer_datetime_format,␣
 ↪keep_date_col, date_parser, date_format, dayfirst, cache_dates, iterator,␣
 ↪chunksize, compression, thousands, decimal, lineterminator, quotechar,␣
 ↪quoting, doublequote, escapechar, comment, encoding, encoding_errors, dialect ␣
 ↪on_bad_lines, delim_whitespace, low_memory, memory_map, float_precision,␣
 ↪storage_options, dtype_backend)
   1013 kwds_defaults = _refine_defaults_read(
   1014     dialect,
   1015     delimiter,
   (…)   1022     dtype_backend=dtype_backend,
   1023 )
   1024 kwds.update(kwds_defaults)
-> 1026 return _read(filepath_or_buffer, kwds)

File ~/miniconda3/envs/ml/lib/python3.11/site-packages/pandas/io/parsers/reader.
 ↪py:620, in _read(filepath_or_buffer, kwds)
    617 _validate_names(kwds.get("names", None))
    619 # Create the parser.
--> 620 parser = TextFileReader(filepath_or_buffer, **kwds)
    622 if chunksize or iterator:
    623     return parser

File ~/miniconda3/envs/ml/lib/python3.11/site-packages/pandas/io/parsers/reader.
 ↪py:1620, in TextFileReader.__init__(self, f, engine, **kwds)
   1617     self.options["has_index_names"] = kwds["has_index_names"]
   1619 self.handles: IOHandles | None = None
-> 1620 self._engine = self._make_engine(f, self.engine)

File ~/miniconda3/envs/ml/lib/python3.11/site-packages/pandas/io/parsers/reader.
 ↪py:1880, in TextFileReader._make_engine(self, f, engine)
   1878     if "b" not in mode:
   1879         mode += "b"
-> 1880 self.handles = get_handle(
   1881     f,
   1882     mode,
   1883     encoding=self.options.get(          , None),
   1884     compression=self.options.get(             , None),
```

```
   1885     memory_map=self.options.get(             , False),
   1886     is_text=is_text,
   1887     errors=self.options.get(                  ,          ),
   1888     storage_options=self.options.get(                  , None),
   1889 )
   1890 assert self.handles is not None
   1891 f = self.handles.handle

File ~/miniconda3/envs/ml/lib/python3.11/site-packages/pandas/io/common.py:728,
 ↪in get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text,␣
 ↪errors, storage_options)
    725     codecs.lookup_error(errors)
    727 # open URLs
--> 728 ioargs = _get_filepath_or_buffer(
    729     path_or_buf,
    730     encoding=encoding,
    731     compression=compression,
    732     mode=mode,
    733     storage_options=storage_options,
    734 )
    736 handle = ioargs.filepath_or_buffer
    737 handles: list[BaseBuffer]

File ~/miniconda3/envs/ml/lib/python3.11/site-packages/pandas/io/common.py:384,
 ↪in _get_filepath_or_buffer(filepath_or_buffer, encoding, compression, mode,␣
 ↪storage_options)
    382 # assuming storage_options is to be interpreted as headers
    383 req_info = urllib.request.Request(filepath_or_buffer,␣
 ↪headers=storage_options)
--> 384 with urlopen(req_info) as req:
    385     content_encoding = req.headers.get("Content-Encoding", None)
    386     if content_encoding == "gzip":
    387         # Override compression based on Content-Encoding header

File ~/miniconda3/envs/ml/lib/python3.11/site-packages/pandas/io/common.py:289,
 ↪in urlopen(*args, **kwargs)
    283 """
    284 Lazy-import wrapper for stdlib urlopen, as that imports a big chunk of
    285 the stdlib.
    286 """
    287 import urllib.request
--> 289 return urllib.request.urlopen(*args, **kwargs)

File ~/miniconda3/envs/ml/lib/python3.11/urllib/request.py:216, in urlopen(url,
 ↪data, timeout, cafile, capath, cadefault, context)
    214 else:
    215     opener = _opener
--> 216 return opener.open(url, data, timeout)
```

```
File ~/miniconda3/envs/ml/lib/python3.11/urllib/request.py:525, in␣
↪OpenerDirector.open(self, fullurl, data, timeout)
    523 for processor in self.process_response.get(protocol, []):
    524     meth = getattr(processor, meth_name)
--> 525     response = meth(req, response)
    527 return response

File ~/miniconda3/envs/ml/lib/python3.11/urllib/request.py:634, in␣
↪HTTPErrorProcessor.http_response(self, request, response)
    631 # According to RFC 2616, "2xx" code indicates that the client's
    632 # request was successfully received, understood, and accepted.
    633 if not (200 <= code < 300):
--> 634     response = self.parent.error(
    635                 , request, response, code, msg, hdrs)
    637 return response

File ~/miniconda3/envs/ml/lib/python3.11/urllib/request.py:563, in␣
↪OpenerDirector.error(self, proto, *args)
    561 if http_err:
    562     args = (dict, 'default', 'http_error_default') + orig_args
--> 563     return self._call_chain(*args)

File ~/miniconda3/envs/ml/lib/python3.11/urllib/request.py:496, in␣
↪OpenerDirector._call_chain(self, chain, kind, meth_name, *args)
    494 for handler in handlers:
    495     func = getattr(handler, meth_name)
--> 496     result = func(*args)
    497     if result is not None:
    498         return result

File ~/miniconda3/envs/ml/lib/python3.11/urllib/request.py:643, in␣
↪HTTPDefaultErrorHandler.http_error_default(self, req, fp, code, msg, hdrs)
    642 def http_error_default(self, req, fp, code, msg, hdrs):
--> 643     raise HTTPError(req.full_url, code, msg, hdrs, fp)

HTTPError: HTTP Error 403: Forbidden
```

[76]:
```python
import pandas as pd

# Load your local CeNGEN-mapped expression file
mapping_df = pd.read_csv("data/processed/csv/gene_expr_CeNGEN_base_mapped.csv")

# Inspect columns to identify mapping columns
print(mapping_df.columns[:10])  # just to verify the header structure
display(mapping_df.head(5))
```

```
Index(['Unnamed: 0', 'WBGene00010957', 'WBGene00010958', 'WBGene00014454',
       'WBGene00010959', 'WBGene00010960', 'WBGene00010961', 'WBGene00000829',
       'WBGene00010962', 'WBGene00010963'],
      dtype='object')
```

```
  Unnamed: 0  WBGene00010957  WBGene00010958  WBGene00014454  WBGene00010959  \
0       AINL      232.125000       26.125000       18.125000       18.375000
1       AINR      232.125000       26.125000       18.125000       18.375000
2       ASIR      215.166667       21.333333       17.833333        7.833333
3       ASJR      873.666667       71.222222       69.555556       37.222222
4        PVT      350.000000       26.000000       29.000000        8.000000

   WBGene00010960  WBGene00010961  WBGene00000829  WBGene00010962  \
0       77.750000        5.375000       18.875000      143.125000
1       77.750000        5.375000       18.875000      143.125000
2       53.000000        4.666667       14.333333       50.833333
3       47.555556        4.222222       18.222222      168.666667
4      113.000000        6.000000       22.000000       65.000000

   WBGene00010963  …  WBGene00021598  WBGene00021597  WBGene00021596  \
0       14.000000  …        0.000000        0.875000        0.250000
1       14.000000  …        0.000000        0.875000        0.250000
2       17.500000  …        0.166667        1.166667        0.500000
3       20.111111  …        0.000000        3.444444        1.444444
4       33.000000  …        0.000000        2.000000        2.000000

   WBGene00021595  WBGene00021594  WBGene00219753  WBGene00201093  \
0        0.000000        0.000000        0.000000             0.0
1        0.000000        0.000000        0.000000             0.0
2        0.666667        0.000000        0.500000             0.0
3        0.555556        0.111111        0.777778             0.0
4        0.000000        0.000000        0.000000             0.0

   WBGene00007064  WBGene00197051  WBGene00199940
0        0.125000             0.0             0.0
1        0.125000             0.0             0.0
2        1.166667             0.0             0.0
3        2.000000             0.0             0.0
4        4.000000             0.0             0.0

[5 rows x 22470 columns]
```

[78]:
```python
import pandas as pd, gzip, io, requests

# Latest release (update WS290 to newer if needed)
url = "https://ftp.wormbase.org/pub/wormbase/species/c_elegans/annotation/
 ↪c_elegans.PRJNA13758.WS290.geneIDs.txt.gz"
```

```
print("Downloading mapping from WormBase...")
r = requests.get(url, timeout=120)
r.raise_for_status()

with gzip.open(io.BytesIO(r.content), "rt") as f:
    mapping = pd.read_csv(f, sep="\t", header=None,␣
 ↪names=["WBGeneID","SeqName","GeneName"])

print("  Mapping loaded:", mapping.shape)
display(mapping.head())

# Merge with your stability table
mapped = stable_tbl.copy()
mapped.index.name = "WBGeneID"
mapped = mapped.reset_index().merge(mapping[["WBGeneID","GeneName"]],␣
 ↪on="WBGeneID", how="left")
mapped = mapped.set_index("GeneName").drop(columns=["WBGeneID"])
display(mapped.head(20))
```

```
Downloading mapping from WormBase…
```

```
---------------------------------------------------------------------------
HTTPError                                 Traceback (most recent call last)
Cell In[78], line 8
      6 print("Downloading mapping from WormBase…")
      7 r = requests.get(url, timeout=120)
----> 8 r.raise_for_status()
     10 with gzip.open(io.BytesIO(r.content), "rt") as f:
     11     mapping = pd.read_csv(f, sep="\t", header=None,␣
 ↪names=["WBGeneID","SeqName","GeneName"])

File ~/miniconda3/envs/ml/lib/python3.11/site-packages/requests/models.py:1026,
 ↪in Response.raise_for_status(self)
   1021     http_error_msg = (
   1022         f"{self.status_code} Server Error: {reason} for url: {self.url}
   1023     )
   1025 if http_error_msg:
-> 1026     raise HTTPError(http_error_msg, response=self)

HTTPError: 403 Client Error: Forbidden for url: https://ftp.wormbase.org/pub/
 ↪wormbase/species/c_elegans/annotation/c_elegans.PRJNA13758.WS290.geneIDs.txt.gz
```

[79]:
```
import pandas as pd, gzip, io, requests

url = "https://ftp.wormbase.org/pub/wormbase/species/c_elegans/annotation/
 ↪c_elegans.PRJNA13758.WS290.geneIDs.txt.gz"
```

```
headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/122.
 ↪0 Safari/537.36"}

print("Downloading mapping from WormBase with spoofed agent...")
r = requests.get(url, headers=headers, timeout=120)
r.raise_for_status()

with gzip.open(io.BytesIO(r.content), "rt") as f:
    mapping = pd.read_csv(f, sep="\t", header=None,␣
 ↪names=["WBGeneID","SeqName","GeneName"])

print("  Mapping loaded:", mapping.shape)
display(mapping.head())

# merge with your stability table
mapped = stable_tbl.copy()
mapped.index.name = "WBGeneID"
mapped = mapped.reset_index().merge(mapping[["WBGeneID","GeneName"]],␣
 ↪on="WBGeneID", how="left")
mapped = mapped.set_index("GeneName").drop(columns=["WBGeneID"])
display(mapped.head(20))
```

```
Downloading mapping from WormBase with spoofed agent…
```

```
---------------------------------------------------------------------------
HTTPError                                 Traceback (most recent call last)
Cell In[79], line 9
      7 print("Downloading mapping from WormBase with spoofed agent…")
      8 r = requests.get(url, headers=headers, timeout=120)
----> 9 r.raise_for_status()
     11 with gzip.open(io.BytesIO(r.content), "rt") as f:
     12     mapping = pd.read_csv(f, sep="\t", header=None,␣
 ↪names=["WBGeneID","SeqName","GeneName"])

File ~/miniconda3/envs/ml/lib/python3.11/site-packages/requests/models.py:1026,
 ↪in Response.raise_for_status(self)
   1021     http_error_msg = (
   1022         f"{self.status_code} Server Error: {reason} for url: {self.url}
   1023     )
   1025 if http_error_msg:
-> 1026     raise HTTPError(http_error_msg, response=self)

HTTPError: 403 Client Error: Forbidden for url: https://ftp.wormbase.org/pub/
 ↪wormbase/species/c_elegans/annotation/c_elegans.PRJNA13758.WS290.geneIDs.txt.gz
```

[82]:
```python
from Bio import SeqIO

fasta_path = "caenorhabditis_elegans.PRJNA13758.WBPS19.genomic.fa"

# Count and preview first few records
records = SeqIO.parse(fasta_path, "fasta")
for i, rec in enumerate(records):
    print(f"{i+1:3d}. {rec.id:30s} length={len(rec.seq)}")
    if i >= 9:
        break
```

```
  1. I                              length=15072434
  2. II                             length=15279421
  3. III                            length=13783801
  4. IV                             length=17493829
  5. V                              length=20924180
  6. X                              length=17718942
  7. MtDNA                          length=13794
```

[ ]:
```python

```